Visión Artificial

Tema 3. Procesamiento de imagen para visión por computador

Pedro E. López de Teruel Alcolea
Departamento de Ingeniería y Tecnología de Computadores

Universidad de Murcia



Máster en Inteligencia Artificial
2026/2027

- Una imagen es una función de R^2 a R^M:

$$f : \underbrace{[a,b] \times [c,d]}_{\text{Dominio } X \times Y} \rightarrow \underbrace{[0,255]}_{\text{Rango: luminancia}}$$

- Imágenes discretas $\rightarrow$ muestreadas en un *grid* regular.
- Está definida sobre un **dominio** rectangular, y el **rango** es finito.
- P. e., $f(x,y)$ da la luminancia ($M = 1$) en la posición (x,y).
- Para una imagen a color, $M = 3$:

$$f(x,y) = \begin{bmatrix} r(x,y) \\ g(x,y) \\ b(x,y) \end{bmatrix}$$

- Objetivo (inmediato) del filtrado:
  - Formar una **nueva imagen** cuyos píxeles se obtienen a partir de una cierta combinación de los píxeles de la(s) imagen(es) de entrada.

- Objetivos (finales) del filtrado:
  - Extraer **información útil** para procesos de interpretación / *matching* posteriores:
    - *Features*:
      - *Hand engineered*: bordes, esquinas, *blobs*, etc.
      - Aprendidas: *deep learning*.
  - Modificar o resaltar ciertas propiedades:
    - Superresolución, realce (*enhancement*), eliminación de ruido (*denoising*)...



Original



Incremento de contraste



Cambio en *Hue*



Color quantization



Suavizado (*Blurring*)



Rotación



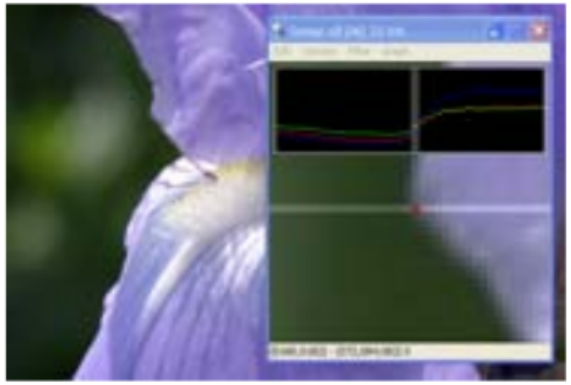
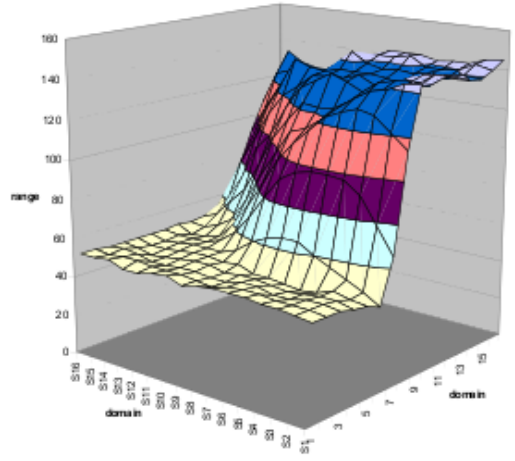


Imagen original / *scanline(s)*

45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120



Array de números por plano y superficie(s) 3D

- Operadores "píxel a píxel", toman una o más imágenes de entrada y generan una de salida:

$$g(\mathbf{x}) = h(f_0(\mathbf{x}), \ldots, f_n(\mathbf{x}))$$

- Dominio discreto $X \times Y$:

$$g(i,j) = h(f(i,j))$$

- Ejemplo lineal (*gain*+*bias*):

$$g(x) = af(x) + b$$

- Ejemplo *blending* (mezcla de dos imágenes):

$$g(x) = (1-\alpha)f_0(x) + \alpha f_1(x)$$

Obsérvense las distintas formas de referirnos a una posición en la imagen: vector $\mathbf{x}$, par fila-columna (i,j), par (x,y), etc.

Procesamiento de imagen para visión artificial
Transformaciones de color



RGB



Red



Green



Blue



Luminancia

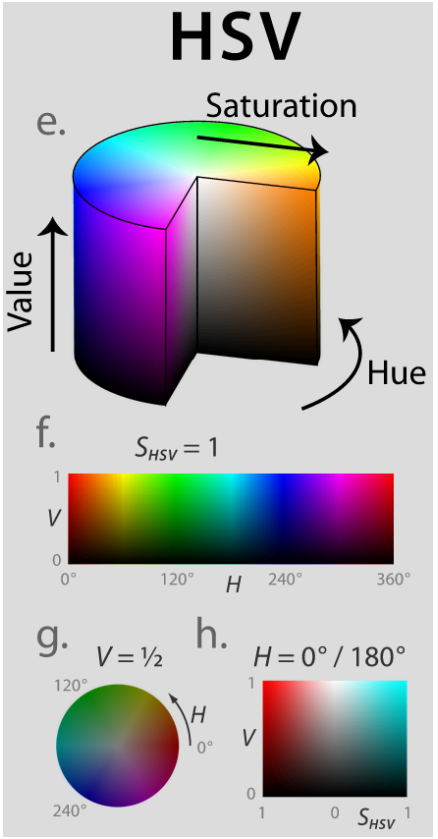


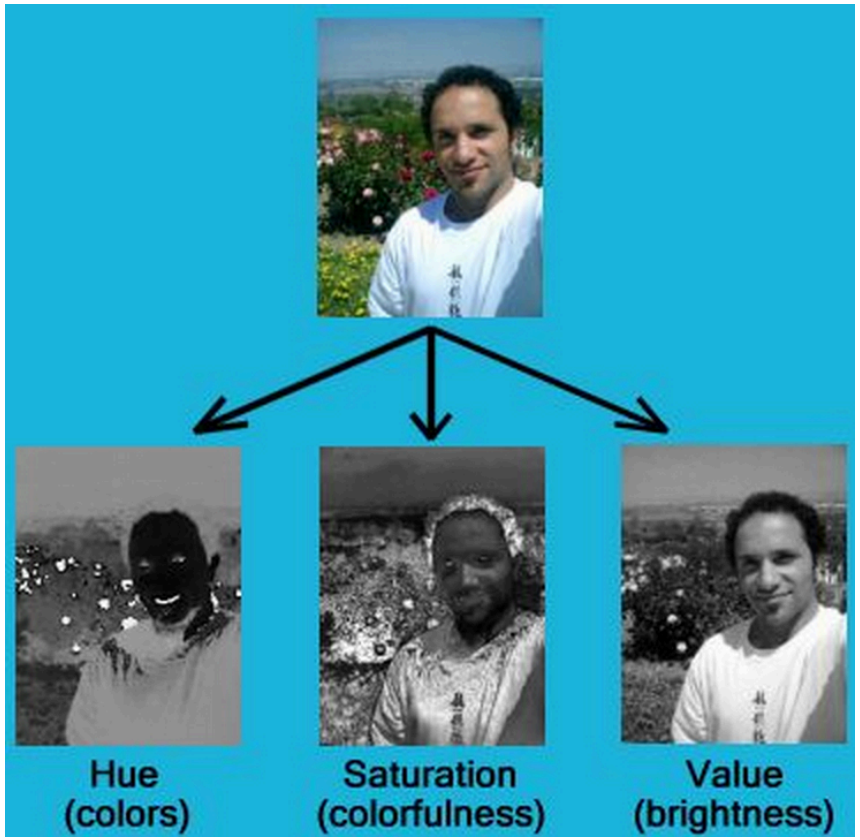
Crominancia 1



Crominancia 2

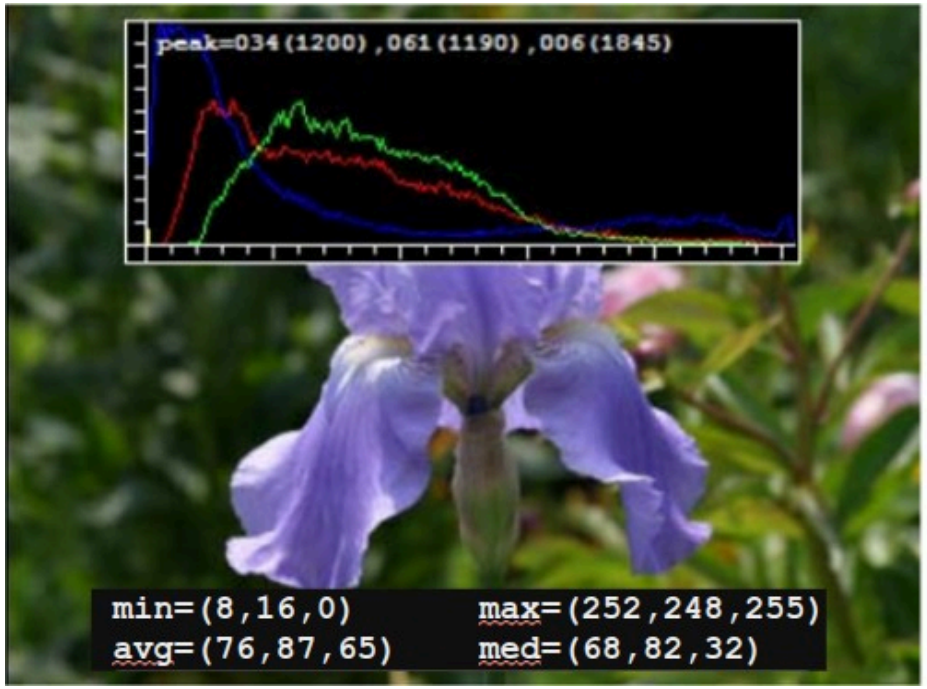
RGB, YUV 420: Y nivel de gris; planos U y V de crominancia, a mitad de resolución que plano Y.



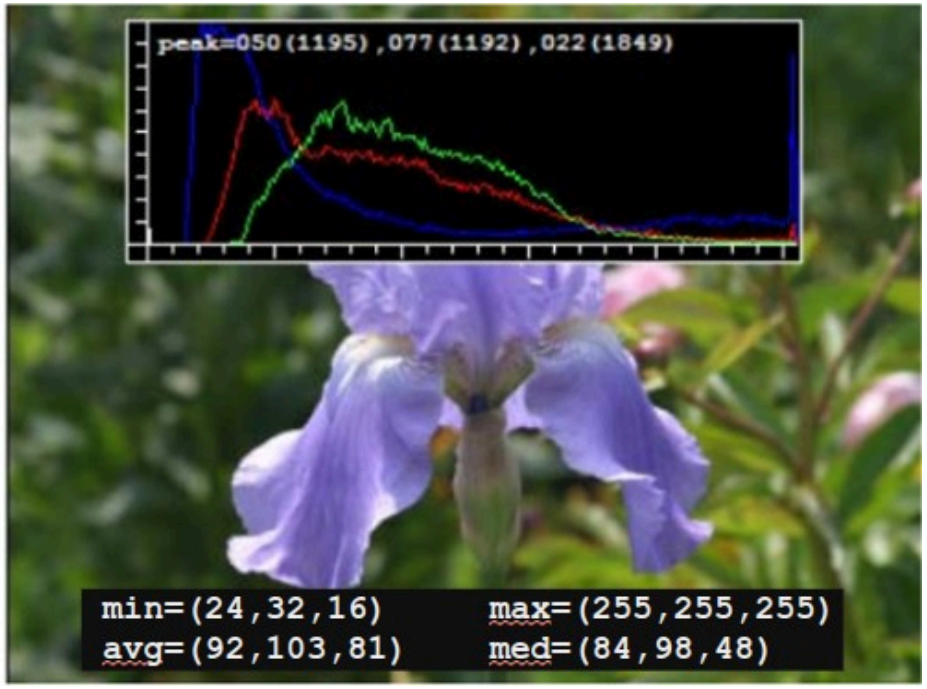


HSV (*Hue, Saturation, Value*) separa "color puro", saturación e intensidad de luz.

Procesamiento de imagen para visión artificial
Incremento de brillo

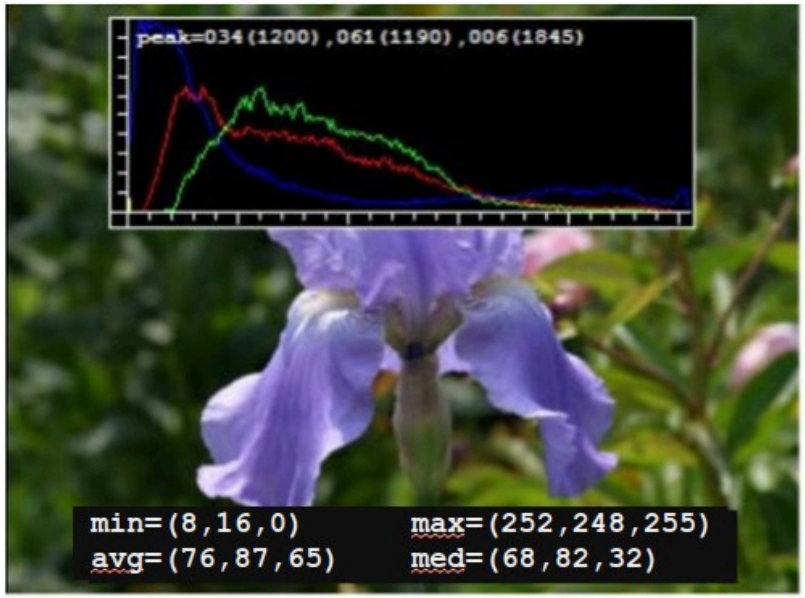


Original

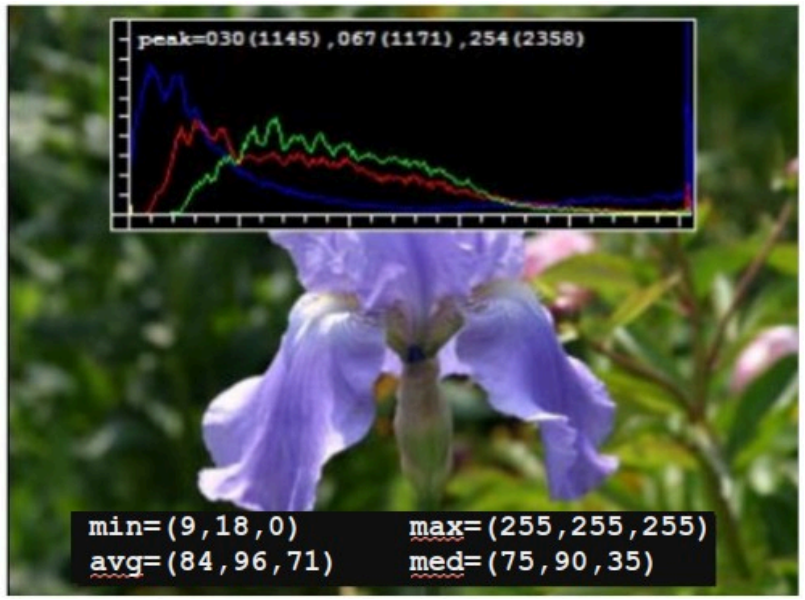


Brillo incrementado

Obsérvese el efecto del incremento de brillo en el histograma (*bias*,+16).



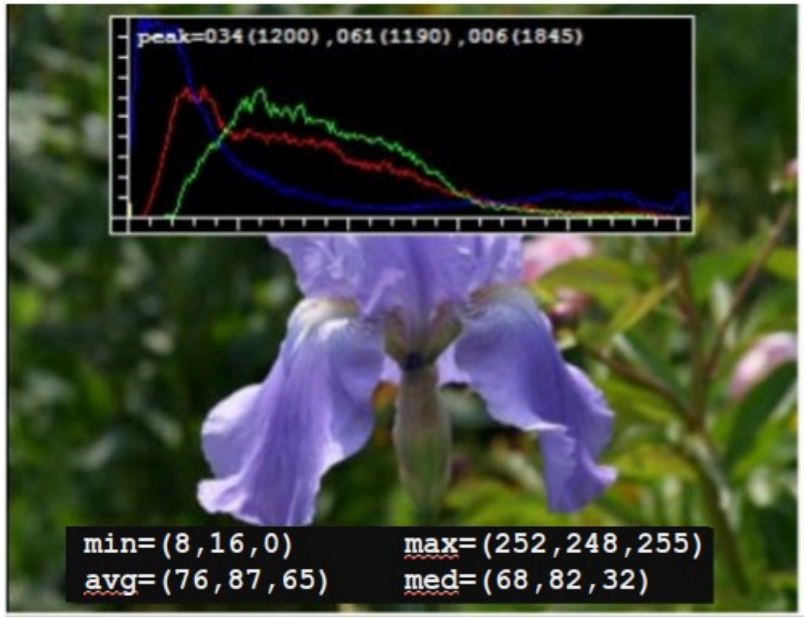
Original



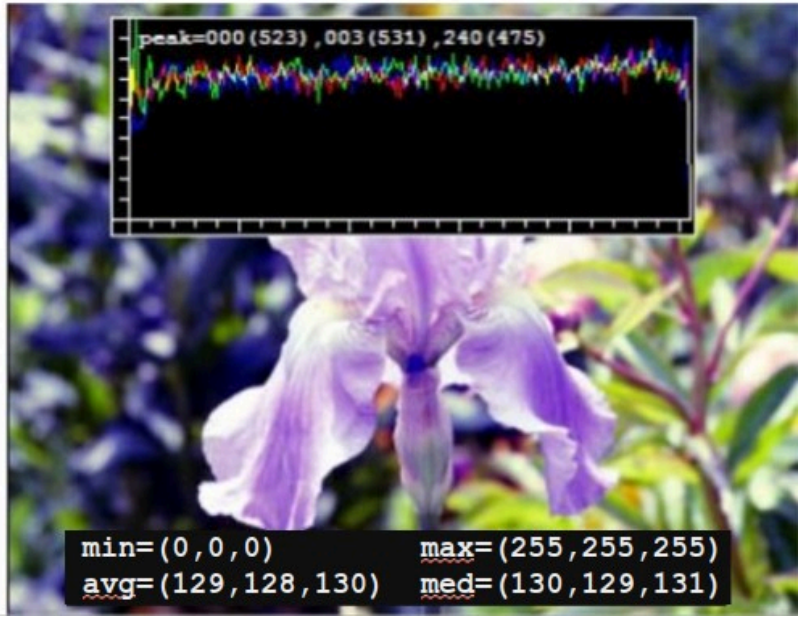
Contraste incrementado

Obsérvese el efecto del incremento de contraste en el histograma (*gain*, $\times 1.1$).

Procesamiento de imagen para visión artificial
Ecualización de histograma



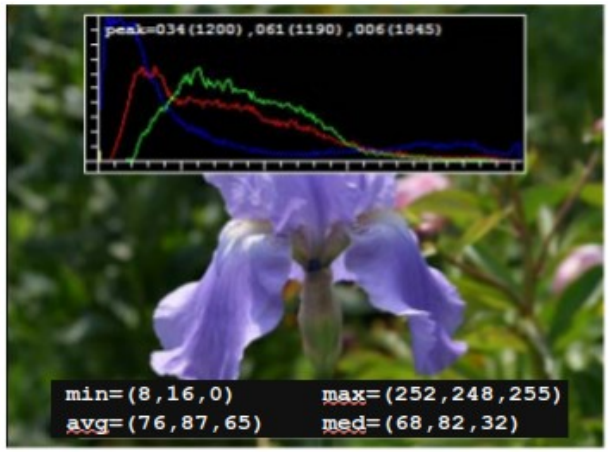
Original

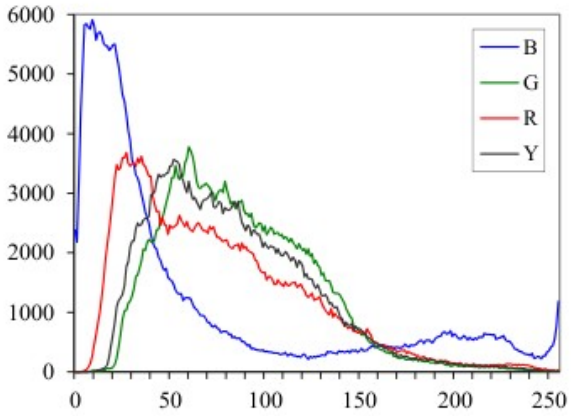


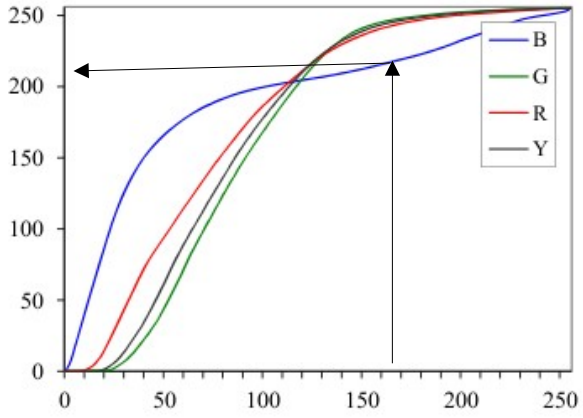
Ecualización de histograma

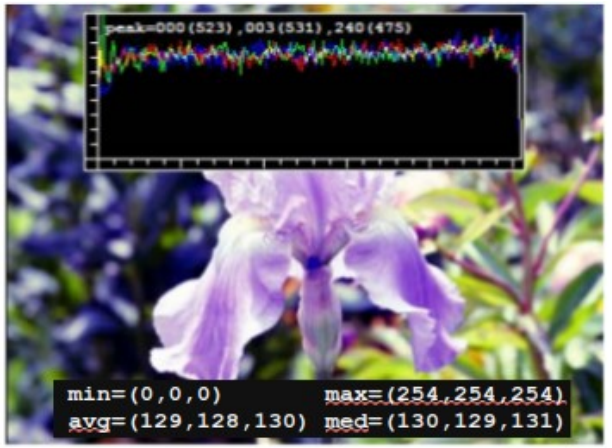
El histograma se modifica para hacer aproximadamente igual la probabilidad de todos los valores R, G y B en el rango [0;255].

Procesamiento de imagen para visión artificial
Ecualización de histograma









Usado como función de transferencia *(c(I)∈[0,1]) x 255*

Histograma acumulado

$$c(I) = \frac{1}{N}\sum_{i=0}^{I} h(i) = c(I-1) + \frac{1}{N}h(I)$$

Imagen original



Histograma original

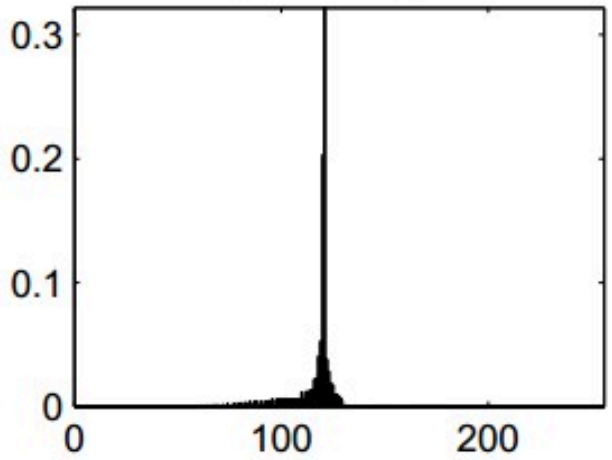
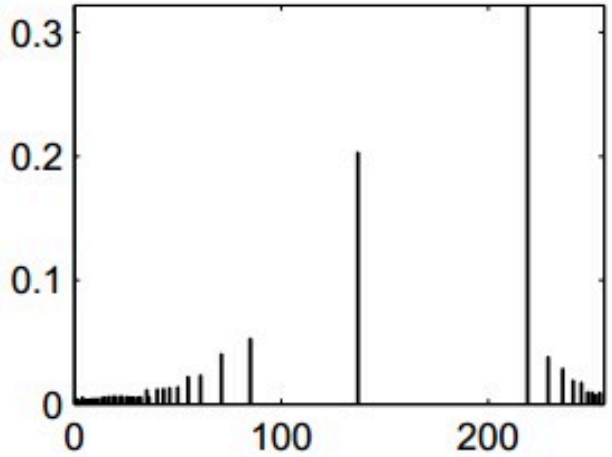


Imagen transformada



Histograma transformado

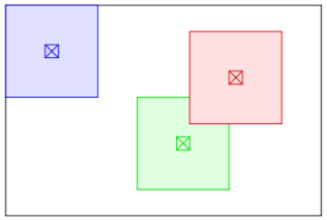


Material para profundizar

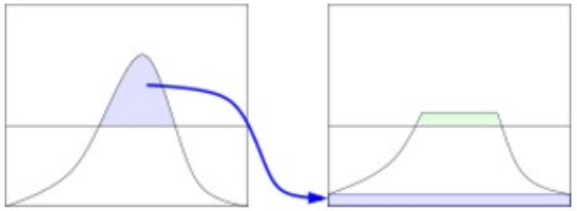
Variante: *(Contrast Limited) Adaptive Histogram Equalization* (**CLAHE**) →

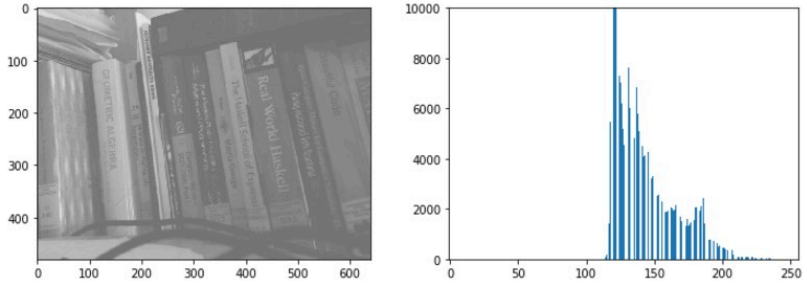
Histogramas locales, cortando picos y repartiendo uniformemente

Histogramas locales → mejoran localidad… pero sobreamplifican contraste en zonas de tono más "plano"

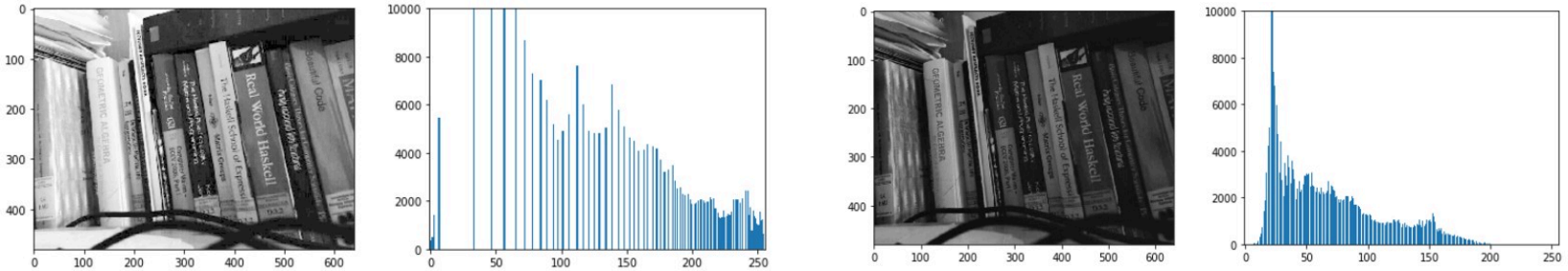


Limitar picos PDF → limita la pendiente de la distribución acumulada





Original



Ecualización normal

Ecualización adaptativa

Procesamiento de imagen para visión artificial
Umbralizado

- Segmentación, habitualmente binarización, de imagen basada en umbralizado simple:

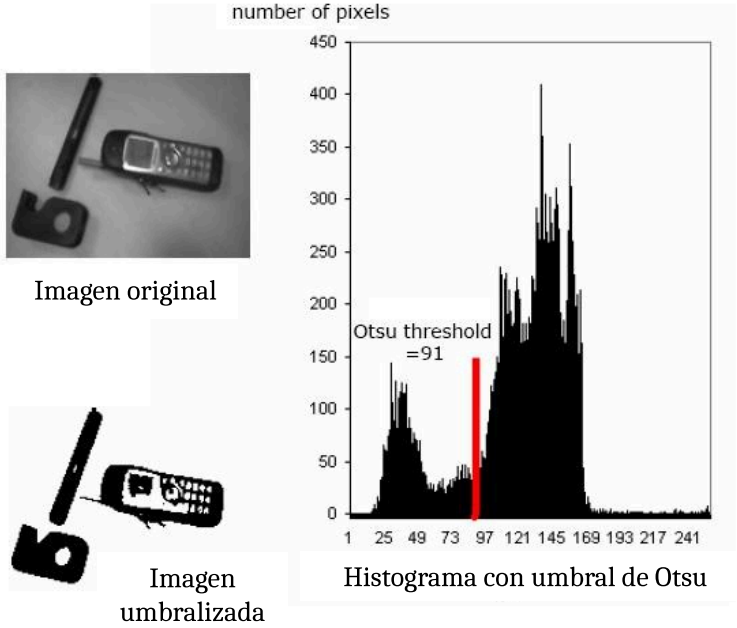




$$g[n,m] = \begin{cases} 1, & f[n,m] > 150 \\ 0, & \text{otherwise.} \end{cases}$$

Procesamiento de imagen para visión artificial
Umbralizado

- Umbralizado de Otsu, adaptado al histograma global:
  - Búsqueda exhaustiva del umbral que minimiza la varianza intraclase, suma ponderada de varianzas de ambas clases:
  - No garantiza una segmentación correcta con sombras o iluminación no uniforme.



Varianza intraclase

t = umbral

$$\sigma_w^2(t) = \omega_1(t)\sigma_1^2(t) + \omega_2(t)\sigma_2^2(t)$$

Varianza clase i=1,2

Prob. clase i=1,2

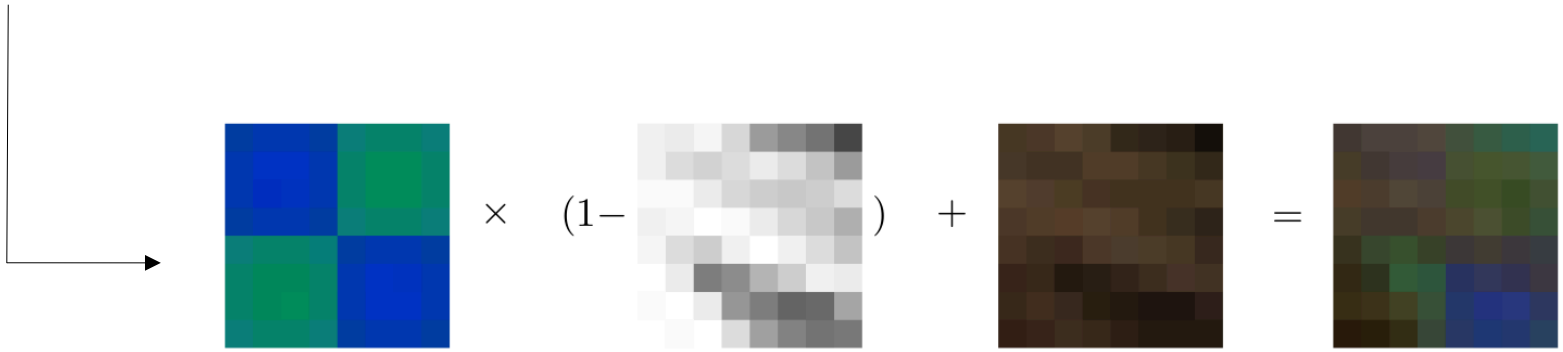
Muchas CNNs producen "imágenes de probabilidad" → También útil en estos casos

https://en.wikipedia.org/wiki/Otsu%27s_method

- Corte de objetos *foreground* para ponerlos en otro *background*:

Con fondo B_1 (F, *in*) | *Matted image* (αF) | Máscara (α) | Con otro fondo B_2 (C, *out*)





Operación de *blending*: $C = (1 - \alpha)B + \alpha F$

- El valor de cada píxel de salida se obtiene a partir de su "vecindad" en la imagen de entrada:



Original



Suavizado (*blurred*)



Realzado



Suavizado con preservación de borde

- Más ejemplos:



Original (binarizada)



Transformada de distancia



Dilatación



Componentes conexas

- Cada píxel se obtiene como combinación lineal de sus vecinos (convolución):

$$g(i,j) = \sum_{k,l} f(i+k, j+l)h(k,l) \longrightarrow g = f \otimes h$$

45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

Original (f)

*

0.1	0.1	0.1
0.1	0.2	0.1
0.1	0.1	0.1

Filtro (h)

=

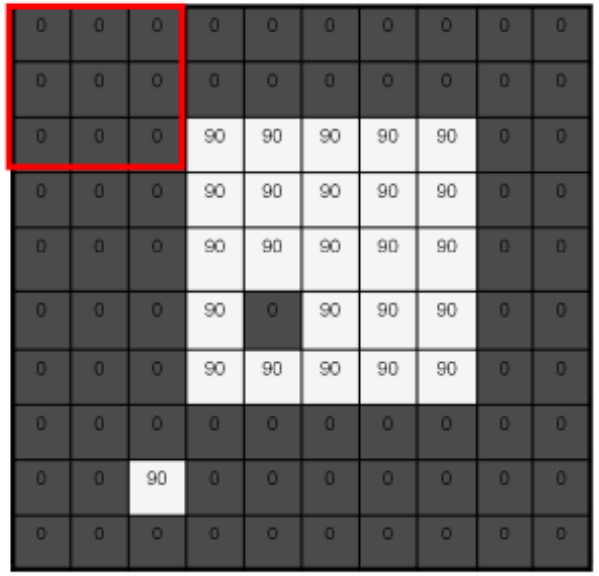
69	95	116	125	129	132
68	92	110	120	126	132
66	86	104	114	124	132
62	78	94	108	120	129
57	69	83	98	112	124
53	60	71	85	100	114

Resultado (g)

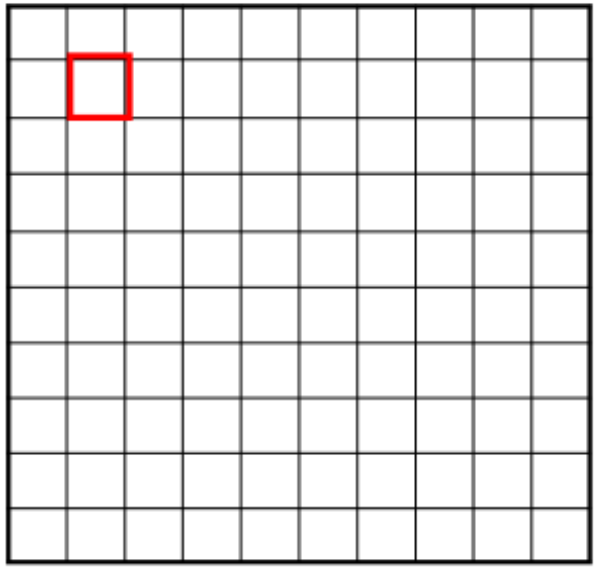
Operación básica en redes neuronales para visión (CNNs)

- Operación, ejemplo filtro de media; a menudo llamado *FilterBox*:

Original



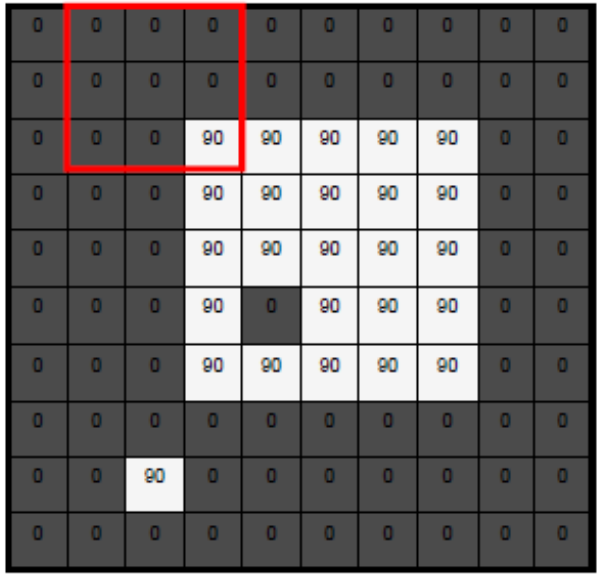
Resultado



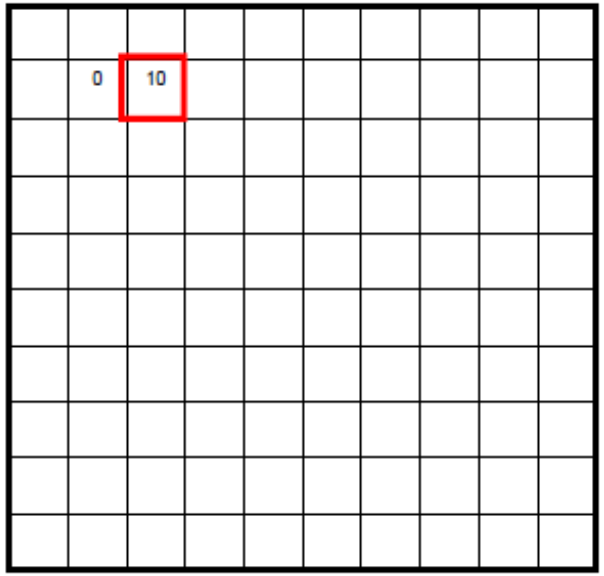
Máscara de convolución aquí: simplemente ventana 3x3 con valores = 1/9

- Operación, ejemplo filtro de media; a menudo llamado *FilterBox*:

Original



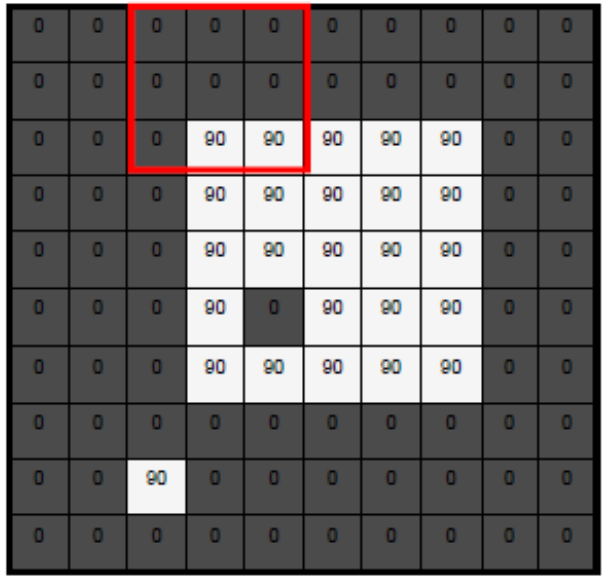
Resultado



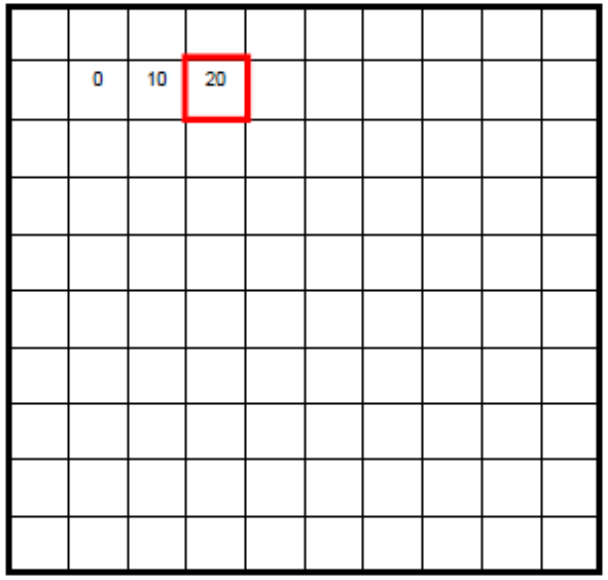
Máscara de convolución aquí: simplemente ventana 3x3 con valores = 1/9

- Operación, ejemplo filtro de media; a menudo llamado *FilterBox*:

Original



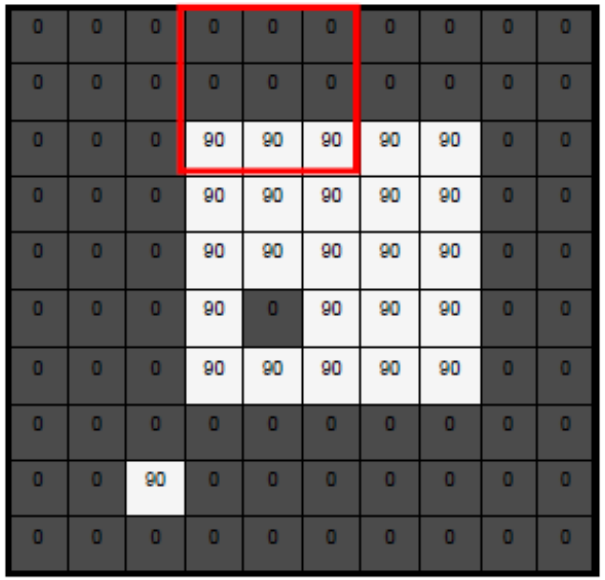
Resultado



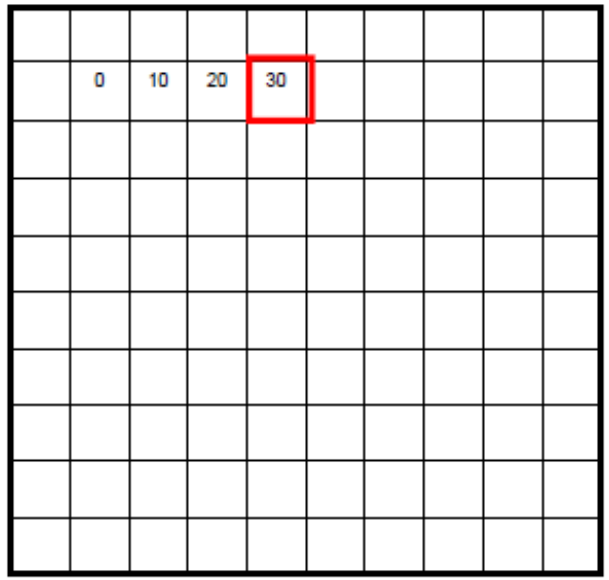
Máscara de convolución aquí: simplemente ventana 3x3 con valores = 1/9

- Operación, ejemplo filtro de media; a menudo llamado *FilterBox*:

Original



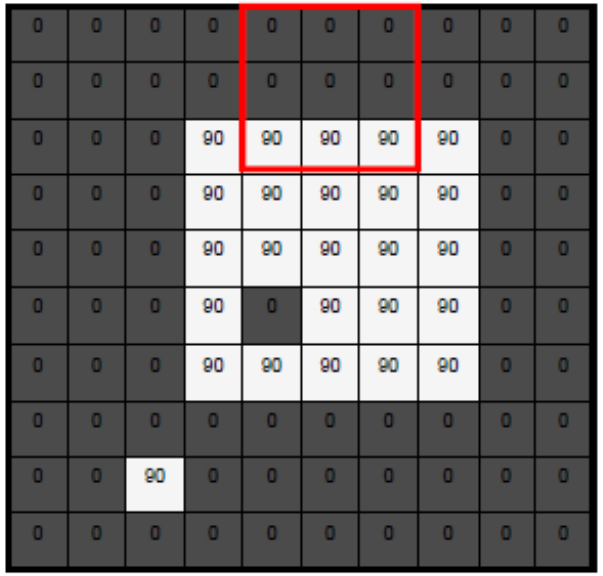
Resultado



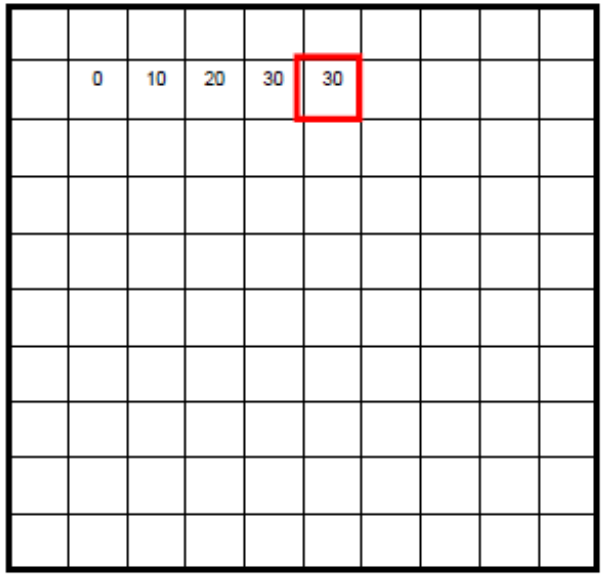
Máscara de convolución aquí: simplemente ventana 3x3 con valores = 1/9

- Operación, ejemplo filtro de media; a menudo llamado *FilterBox*:

Original



Resultado



Máscara de convolución aquí: simplemente ventana 3x3 con valores = 1/9

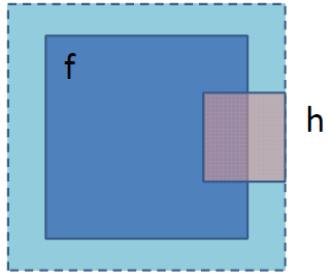
- Resultado final:

Original

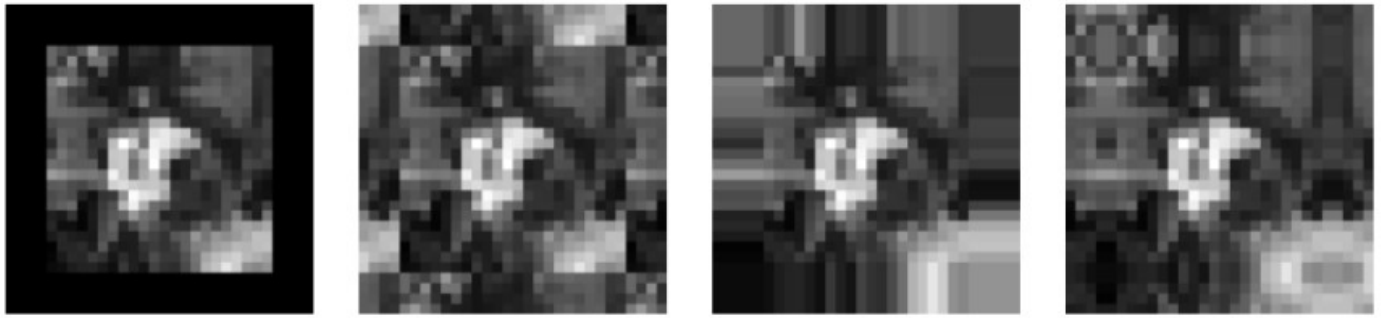
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Resultado

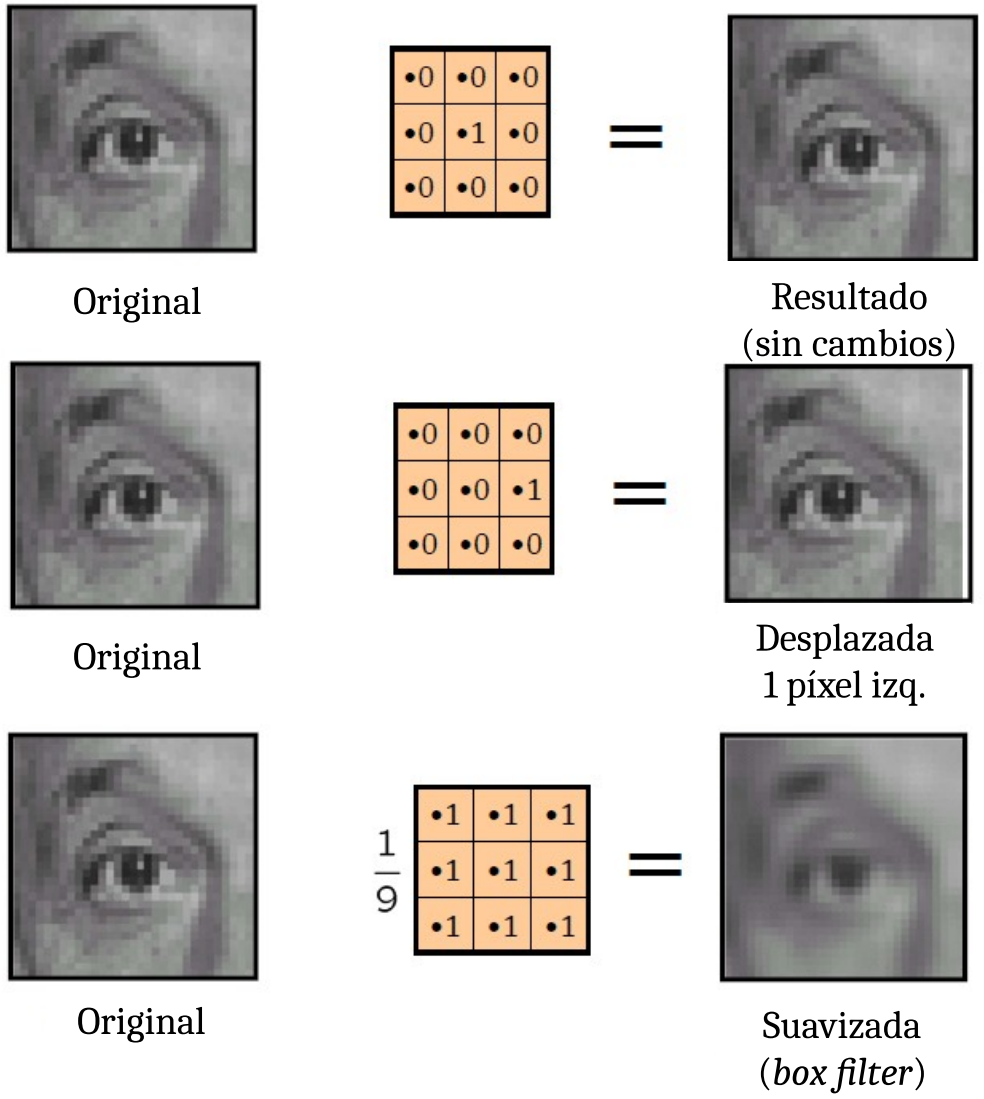
	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	



¡Ojo con los bordes!



Zero *Wrap* *Clamp* *Mirror*


0 0 0
0 1 0
0 0 0
=
Original
Resultado
(sin cambios)
0 0 0
0 0 1
0 0 0
=
Original
Desplazada
1 píxel izq.
1/9
1 1 1
1 1 1
1 1 1
=
Original
Suavizada
(box filter)

Material para profundizar

- Ejemplo FilterBox:
  - Reemplaza el valor de cada píxel con la media de su vecindad 3x3 (8-vecinos).
  - Efecto suavizado: elimina detalles de alta frecuencia.

$$g[n,m] = \frac{1}{9} \sum_{k=n-1}^{n+1} \sum_{l=m-1}^{m+1} f[k,l]$$

$$= \frac{1}{9} \sum_{k=-1}^{1} \sum_{l=-1}^{1} f[n-k, m-l]$$

h

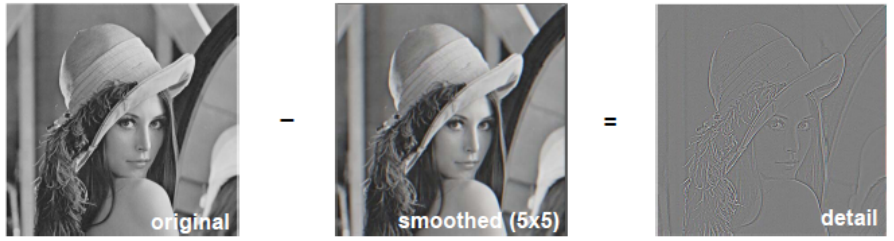
$\frac{1}{9}$

1	1	1
1	1	1
1	1	1



$$(f * h)[m,n] = \frac{1}{9} \sum_{k,l} h[k,l] f[m-k, n-l]$$

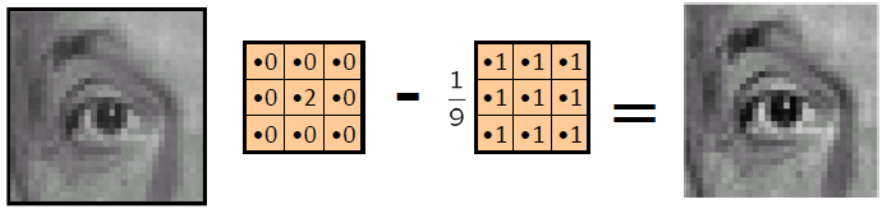
- ¿Qué se lleva el suavizado? → el detalle.



- Si lo volvemos a añadir, recuperamos el realce.



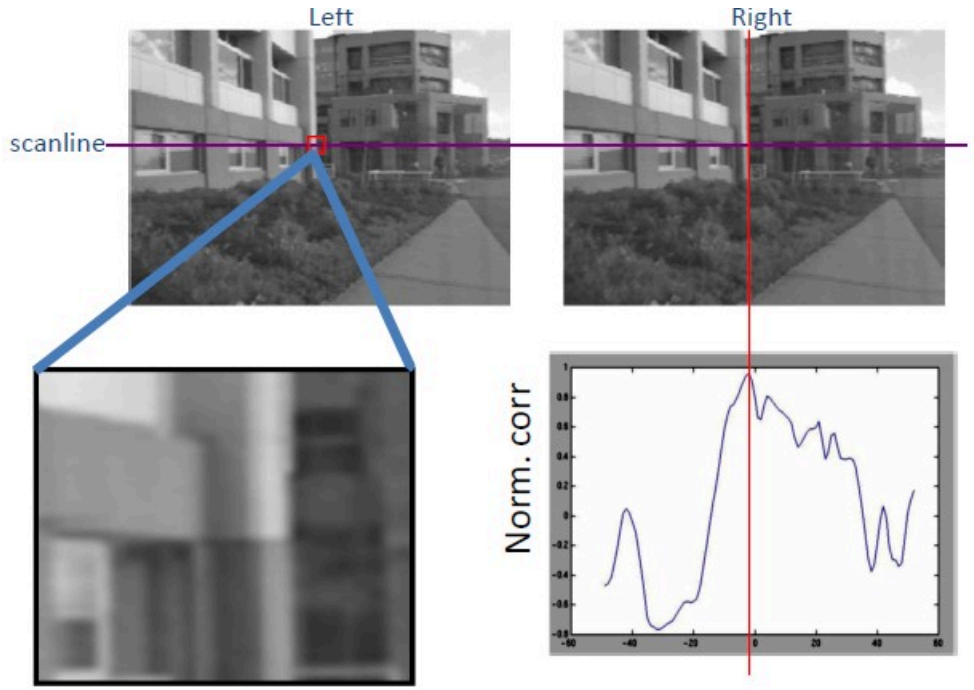
- Filtro *sharpening* (realce):



Procesamiento de imagen para visión artificial

Normalized cross-correlation (NCC)

- Correlación cruzada (*cross-correlation*):
  - Medida de similaridad entre dos señales cuando la una es desplazada sobre la otra; alcanza un máximo cuando hay máxima similaridad.



Ejemplo: búsqueda de correspondencias en un par estéreo calibrado

$$\sum \widetilde{f}[n+k, m+l] * \widetilde{t}[k, l]$$

Se hace normalmente normalizando(*) los parches de la imagen f y la plantilla t (restando la media y dividiendo por la desviación típica):

$$\frac{1}{n}\sum_{x,y}\frac{(f(x,y)-\overline{f})(t(x,y)-\overline{t})}{\sigma_f \sigma_t}$$

(*)También se usa a veces la algo más simple distancia del coseno (NCC y DCC ambos en rango [-1,1]):

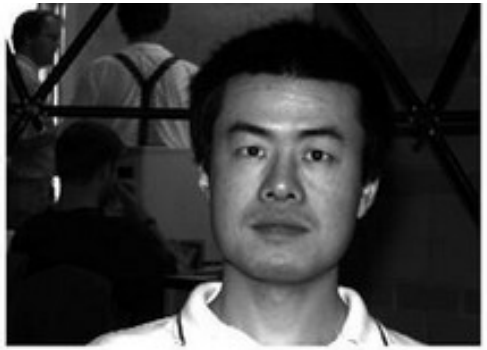
$$\text{cosine similarity} = S_C(A,B) := \cos(\theta) = \frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\cdot\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Procesamiento de imagen para visión artificial

Template matching

- Misma idea que lo anterior, para buscar zonas de imagen que se parezcan a un modelo dado.
  - No es invariante a rotaciones, cambios de escala, etc. (método "ingenuo").

1. Imagen de entrada





2. Plantilla (*template*)

3. NCC (Normalized cross-correlation)





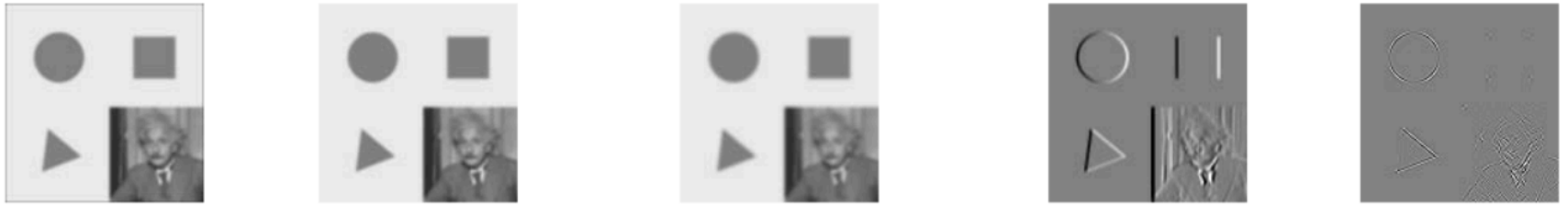
4. Máximo NCC (posición de salida)

Procesamiento de imagen para visión artificial
Filtrado separable

- Un filtro es separable cuando puede reducirse a una pasada por filas y otra por columnas.
- Permite aplicar filtros 2D mediante dos convoluciones 1D.
- **Nota**: Si máscara no simétrica, aún podría expresarse como un *outer product* en algunos casos (ejemplo penúltimo, *Sobel X*: $m = (1,2,1)^T \cdot (-1,0,1)$)

$$\frac{1}{K^2}\begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & 1 & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} \quad \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad \frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix} \quad \frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \frac{1}{4}\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$$

$$\frac{1}{K}\begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix} \quad \frac{1}{4}\begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \quad \frac{1}{16}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix} \quad \frac{1}{2}\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad \frac{1}{2}\begin{bmatrix} 1 & -2 & 1 \end{bmatrix}$$



Material para profundizar

$$g_{\text{sharp}} = f + \gamma(f - h_{\text{blur}} * f)$$

Realce (*sharpening*)

$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Filtro gaussiano (*blurring*)

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Operador laplaciano

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2}\right) G(x, y; \sigma)$$

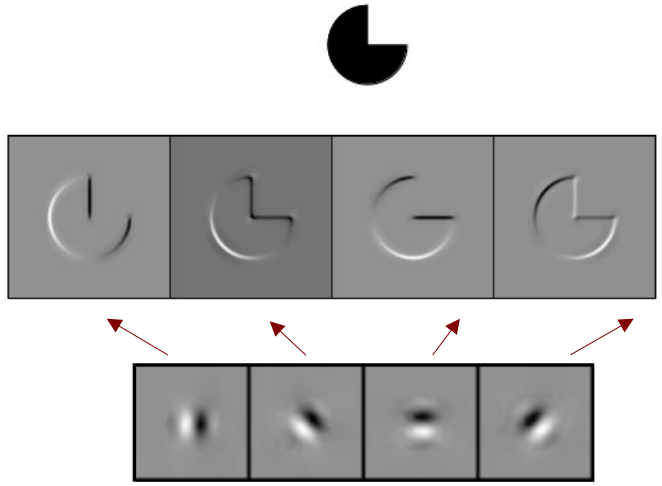
Laplaciana de la *gaussiana*

Importante para SIFT (Tema 4)

$$\hat{\boldsymbol{u}} \cdot \nabla(G * f) = \nabla_{\hat{\boldsymbol{u}}}(G * f) = (\nabla_{\hat{\boldsymbol{u}}} G) * f$$

$$G_{\hat{\boldsymbol{u}}} = uG_x + vG_y = u\frac{\partial G}{\partial x} + v\frac{\partial G}{\partial y}$$

Filtros direccionables (ej. Sobel Operator)



Procesamiento de imagen para visión artificial
Ejemplos numéricos de lo anterior

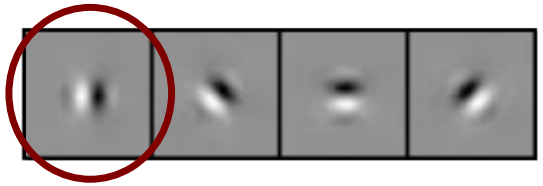
-1	-1	-1
-1	9	-1
-1	-1	-1

Realce (*sharpening*)

-1	-1	-1
-1	8	-1
-1	-1	-1

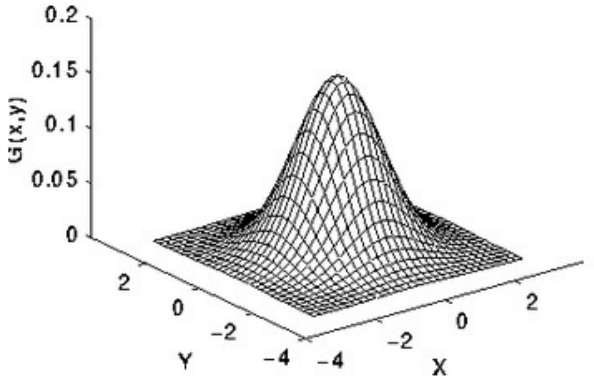
Operador laplaciano

$$S_x = \begin{bmatrix} 1 & 2 & 0 & -2 & -1 \\ 4 & 8 & 0 & -8 & -4 \\ 6 & 12 & 0 & -12 & -6 \\ 4 & 8 & 0 & -8 & -4 \\ 1 & 2 & 0 & -2 & -1 \end{bmatrix}$$



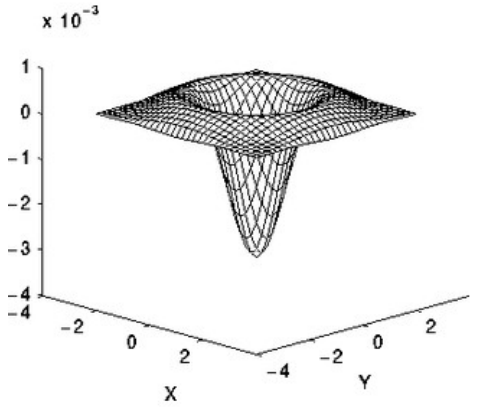
Sobel Operator (en X)

Más común de izq. a der. (signo cambiado)



2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

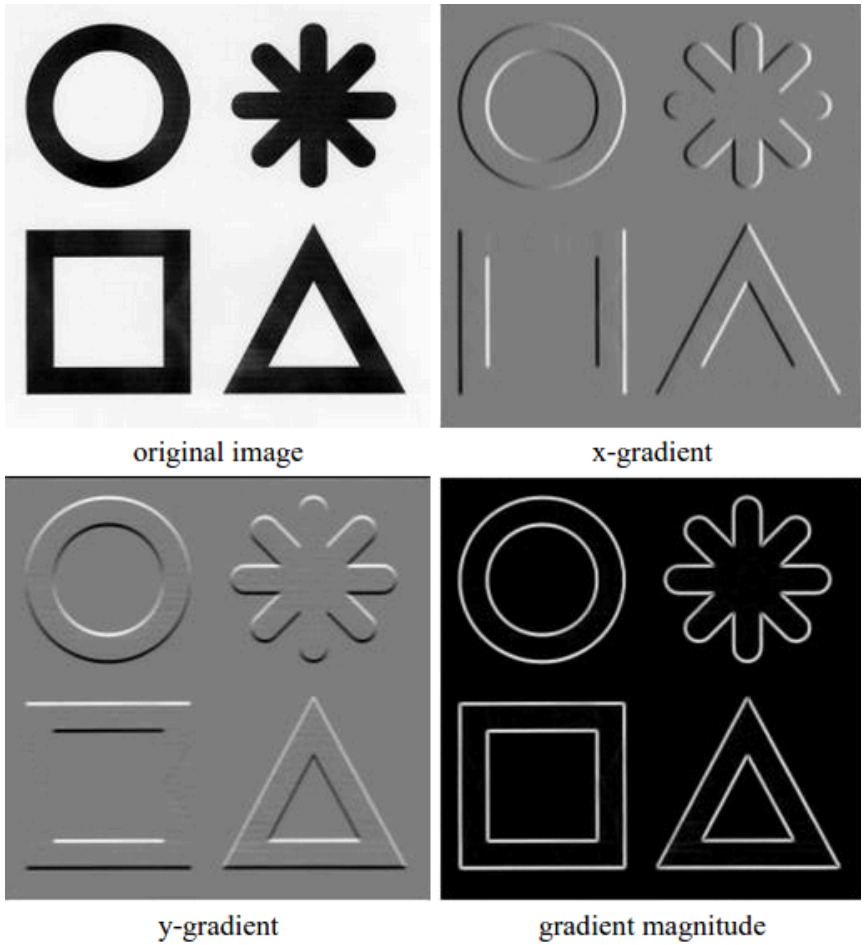
Filtro gaussiano (*blurring*)

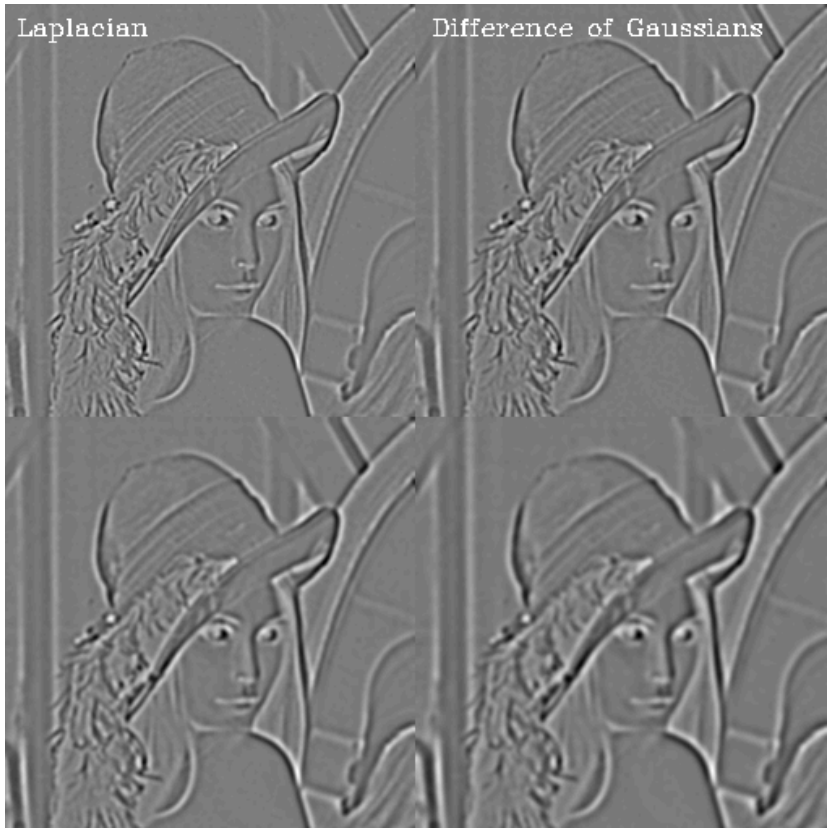


$$\begin{pmatrix} 0 & 0 & 3 & 2 & 2 & 2 & 3 & 0 & 0 \\ 0 & 2 & 3 & 5 & 5 & 5 & 3 & 2 & 0 \\ 3 & 3 & 5 & 3 & 0 & 3 & 5 & 3 & 3 \\ 2 & 5 & 3 & -12 & -23 & -12 & 3 & 5 & 2 \\ 2 & 5 & 0 & -23 & -40 & -23 & 0 & 5 & 2 \\ 2 & 5 & 3 & -12 & -23 & -12 & 3 & 5 & 2 \\ 3 & 3 & 5 & 3 & 0 & 3 & 5 & 3 & 3 \\ 0 & 2 & 3 & 5 & 5 & 5 & 3 & 2 & 0 \\ 0 & 0 & 3 & 2 & 2 & 2 & 3 & 0 & 0 \end{pmatrix}$$

Laplaciana de la gaussiana

Procesamiento de imagen para visión artificial
Ejemplos de filtros lineales aplicados





Volveremos a ellos en el tema 4.

- La salida depende de una combinación no lineal de los vecinos.
  - Por ejemplo, los usados para suavizado *edge-preserving*.



Original



Filtrado Gaussiano
(lineal)



Filtrado de mediana
(no lineal)

Muy utilizado (valor mediano,
en lugar de medio como *FilterBox*)
de los valores de la ventana

- Otras posibilidades para suavizado *edge-preserving* y/o eliminación de ruido *salt & pepper*:

1	2	1	2	4
2	1	3	5	8
1	3	7	6	9
3	4	8	6	7
4	5	7	8	9

Mediana

Se ordenan los valores y nos quedamos con el valor central

1	2	1	2	4
2	1	3	5	8
1	3	7	6	9
3	4	8	6	7
4	5	7	8	9

α-mean

Se descartan los valores menores y mayores (prop. α) y se hace la media de los restantes

	2	1	0	1	2
2	0.1	0.3	0.4	0.3	0.1
1	0.3	0.6	0.8	0.6	0.3
0	0.4	0.8	1.0	0.8	0.4
1	0.3	0.6	0.8	0.6	0.3
2	0.1	0.3	0.4	0.3	0.1

Filtro de dominio

0.0	0.0	0.0	0.0	0.2
0.0	0.0	0.0	0.4	0.8
0.0	0.0	1.0	0.8	0.4
0.0	0.2	0.8	0.8	1.0
0.2	0.4	1.0	0.8	0.4

Filtro de rango

$$d(i,j,k,l) = \exp\left(-\frac{(i-k)^2+(j-l)^2}{2\sigma_d^2}\right)$$

$$r(i,j,k,l) = \exp\left(-\frac{\|f(i,j)-f(k,l)\|^2}{2\sigma_r^2}\right)$$

$$g(i,j) = \frac{\sum_{k,l} f(k,l)w(i,j,k,l)}{\sum_{k,l} w(i,j,k,l)}$$

$$w(i,j,k,l) = \exp\left(-\frac{(i-k)^2+(j-l)^2}{2\sigma_d^2} - \frac{\|f(i,j)-f(k,l)\|^2}{2\sigma_r^2}\right)$$

Material para profundizar

Procesamiento de imagen para visión artificial
Filtrado bilateral

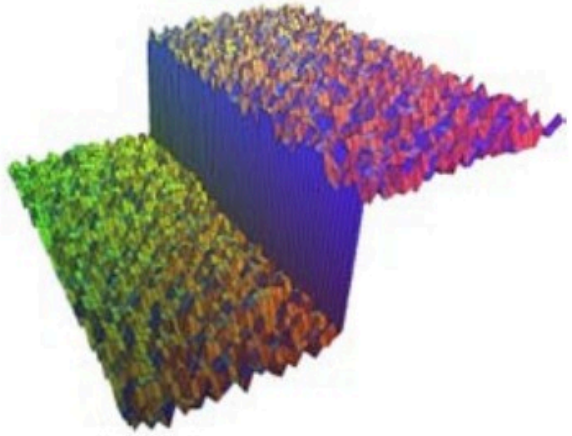
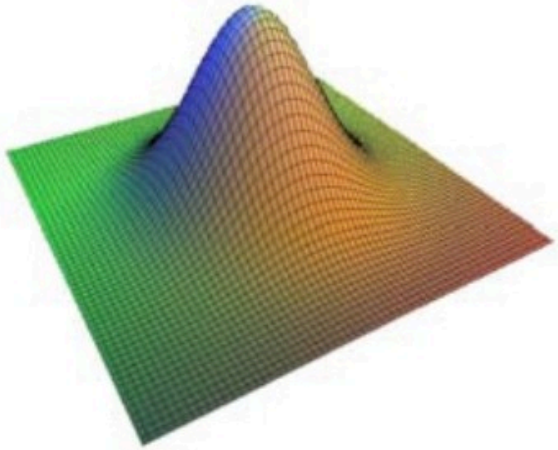
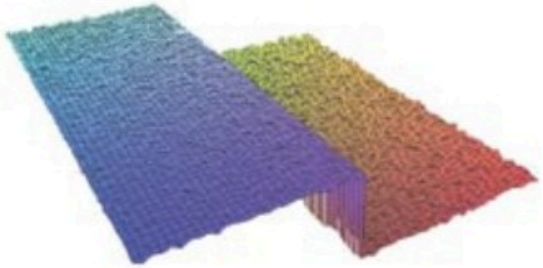


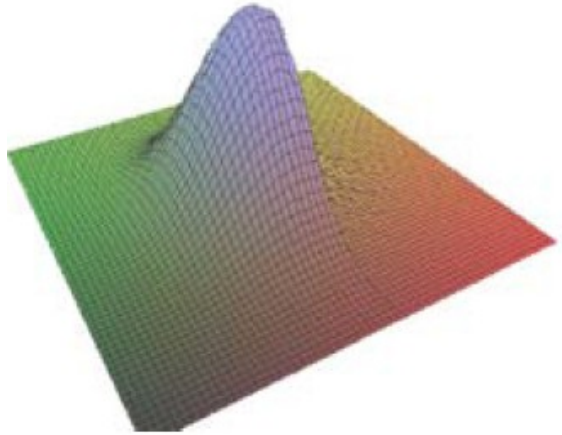
Imagen original

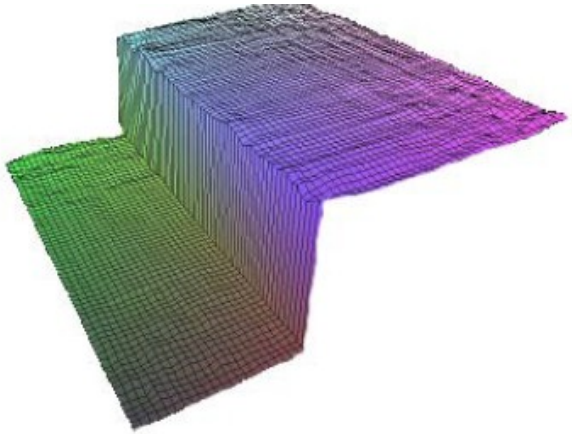


Filtro de dominio



Filtro de rango
(similaridad al pixel central)





Procesamiento de imagen para visión artificial
Ejemplo de filtrado bilateral aplicado

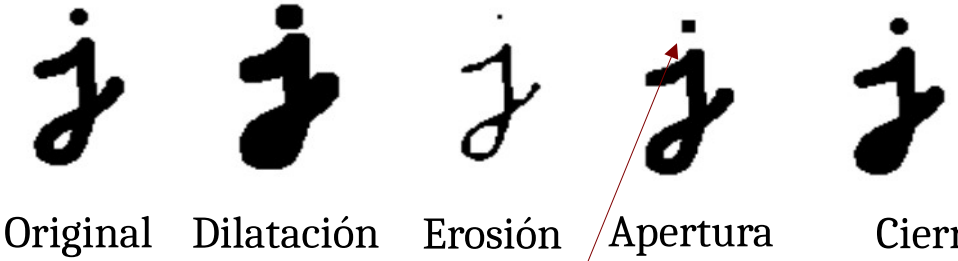


Original Resultado

Sigue eliminando ruido tipo "*salt and pepper*", y respetando los bordes, pero el resultado es más "natural" que en el filtro de mediana

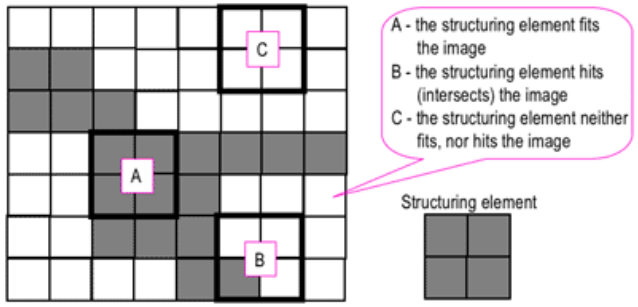
Procesamiento de imagen para visión artificial
Operadores morfológicos

(Consideramos aquí blanco=0, negro=1)



Original Dilatación Erosión Apertura Cierre

Podría haberse eliminado el punto con un elemento estructural más grande



Probing of an image with a structuring element
(white and grey pixels have zero and non-zero values, respectively).

Se suelen aplicar tras *binarizar* la imagen, y la salida es también binaria:

$$\theta(f,t) = \begin{cases} 1 & \text{if } f \geq t, \\ 0 & \text{else,} \end{cases}$$

Cuenta de píxeles activos dentro de la máscara en la imagen de entrada

Umbral

$$\text{dilate}(f,s) = \theta(c,1)$$

$$\text{erode}(f,s) = \theta(c,S)$$

$$\text{open}(f,s) = \text{dilate}(\text{erode}(f,s),s)$$

$$\text{close}(f,s) = \text{erode}(\text{dilate}(f,s),s)$$

c = cuenta de unos dentro del elemento estructural s (normalmente ventana 3x3), de tamaño S=3x3=9

Material para profundizar

- Cada píxel de salida etiquetado con número (color) distinto para cada componente 4- u 8-conectada.
- Ejemplo 4-conectada:

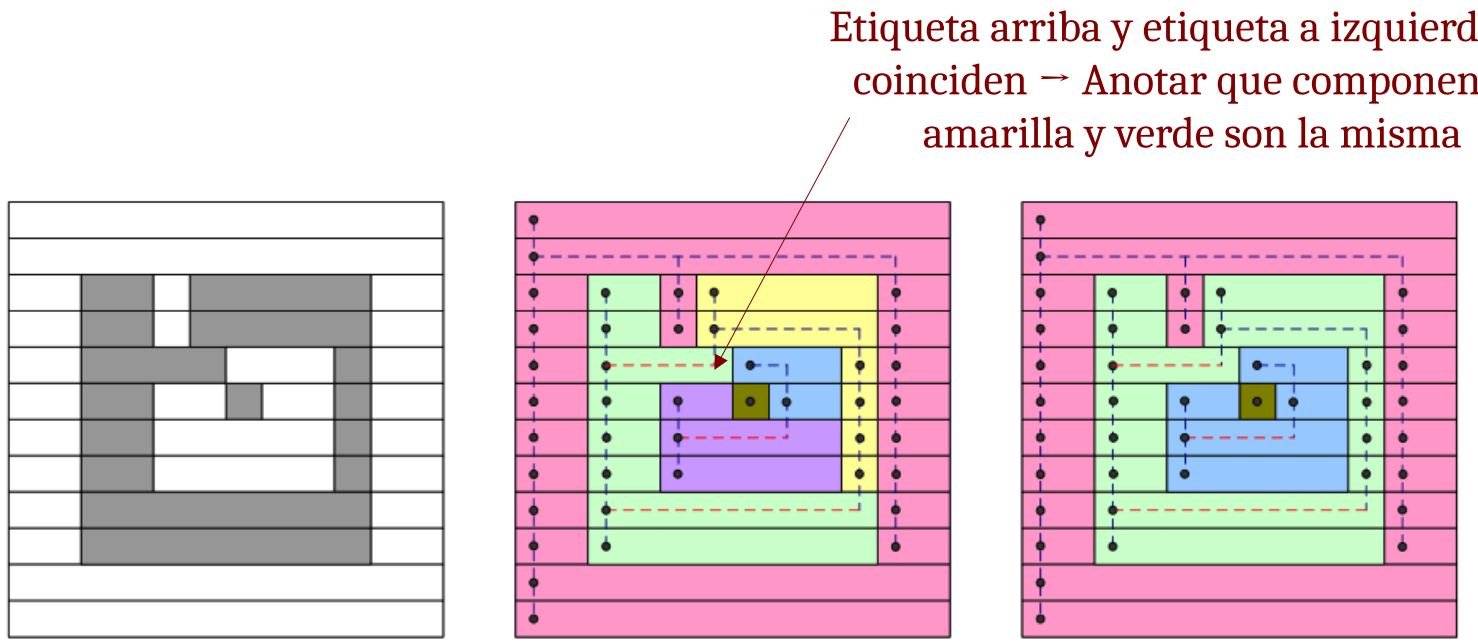


Imagen binaria original

Primera pasada (mirando siempre arriba y a la izquierda) (*conexiones negras*)

Pasada final uniendo componentes no unidos en recorrido anterior (*conexiones rojas*)

- Es habitual calcular después para cada componente su área, centro de masas, *bounding box*, contorno, perímetro, centroide, ...
- ... incluso segundos momentos estadísticos y, a partir de ellos, las direcciones de los ejes principales (vectores principales, *eigensystem*):



$$\boldsymbol{M} = \sum_{(x,y)\in\mathcal{R}} \begin{bmatrix} x-\overline{x} \\ y-\overline{y} \end{bmatrix} \begin{bmatrix} x-\overline{x} & y-\overline{y} \end{bmatrix}$$

$$M = V D V^T$$

Material para profundizar

- Cada píxel de salida almacenará la distancia al píxel activo (aquí $b(k,l) = 0$) de la entrada (imagen binaria b) más cercano:

$$D(i,j) = \min_{k,l:b(k,l)=0} d(i-k, j-l)$$

$$d_1(k,l) = |k| + |l| \qquad d_2(k,l) = \sqrt{k^2 + l^2}$$

Distancia de Manhattan — Distancia euclídea

0	0	0	0	1	0	0
0	0	1	1	1	0	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	0	0	0
0	0	1	0	0	0	0
0	0	0	0	0	0	0

Imagen binaria original ***b***
(aquí "activo" = 0)

0	0	0	0	1	0	0
0	0	1	1	2	0	0
0	1	2	2	3	1	0
0	1	2	3			

Mitad primera pasada
(hacia adelante)

0	0	0	0	1	0	0
0	0	1	1	2	0	0
0	1	2	2	3	1	0
0	1	2	2	1	1	0
0	1	2	1	0	0	0
0	0	1	0	0	0	0
0	0	0	0	0	0	0

Mitad segunda pasada
(hacia atrás)

0	0	0	0	1	0	0
0	0	1	1	1	0	0
0	1	2	2	2	1	0
0	1	2	2	1	1	0
0	1	2	1	0	0	0
0	0	1	0	0	0	0
0	0	0	0	0	0	0

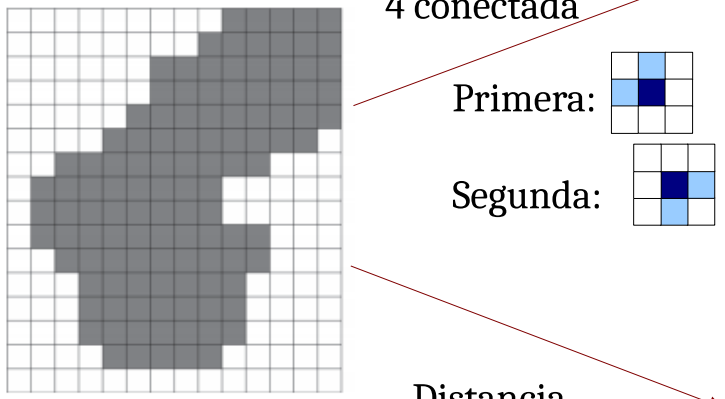
Imagen resultado final ***D***
(usando tipo distancia d_1)

Material para profundizar

Cada pixel no cero se reemplaza por mínimo de 1 más la distancia de vecino N o W

Idem, pero mínimo de valor actual, 1+dist(S),1+dist(E)

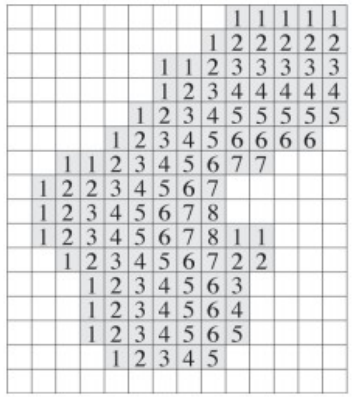
- Ejemplo completo:

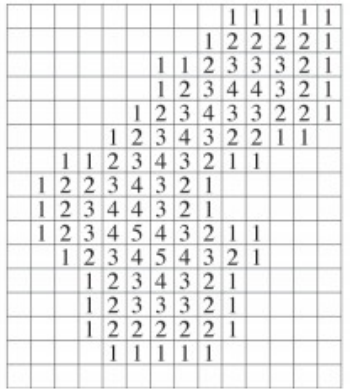


Distancia Manhattan 4 conectada

Primera:

Segunda:





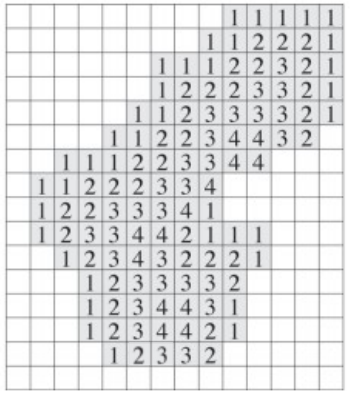
Distancia Manhattan 8 conectada

Primera:

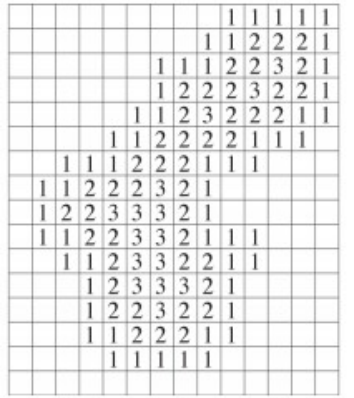


Segunda:



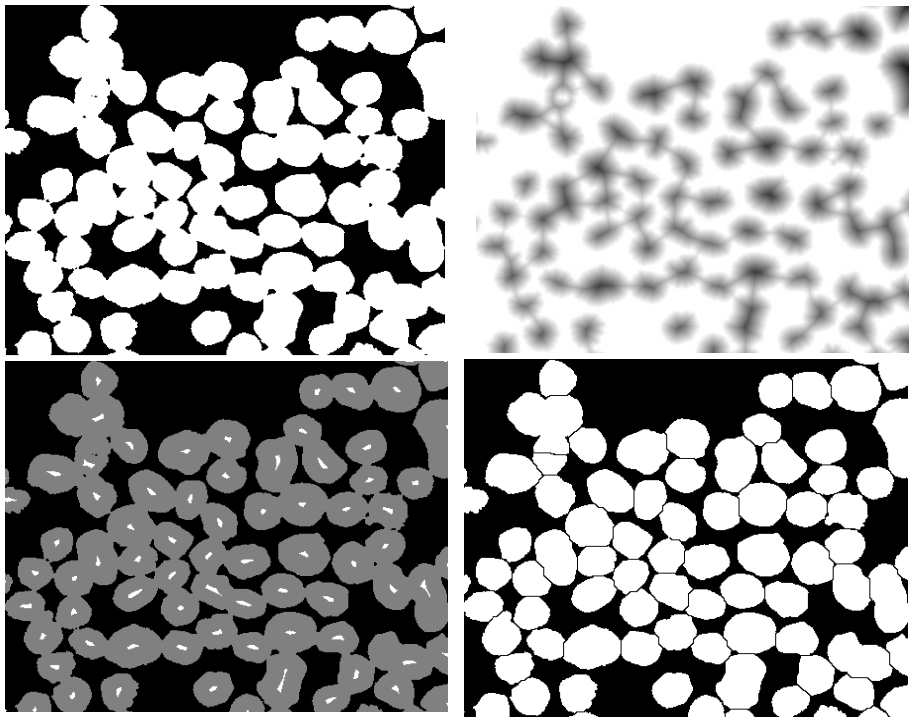


Tras la primera pasada

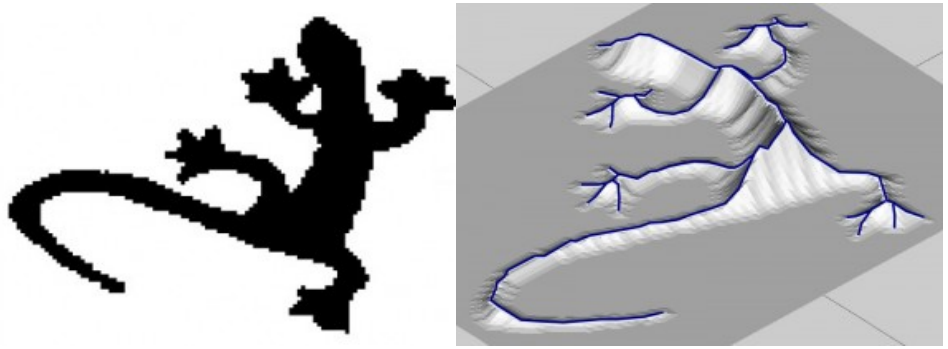


Tras la segunda pasada (final)

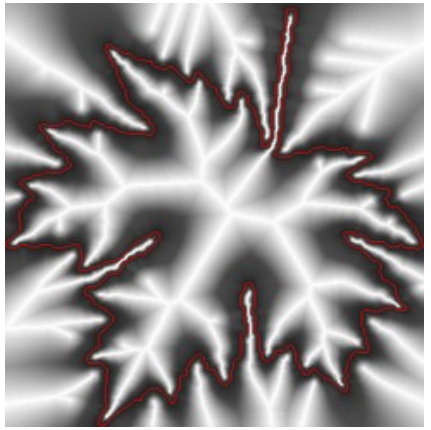
Procesamiento de imagen para visión artificial
Ejemplos de aplicación de la DT



Conteo / separación de células
(máximos locales de la DT +
usar etiquetas de píxeles “fuente” en una
segunda DT a dichos máximos)



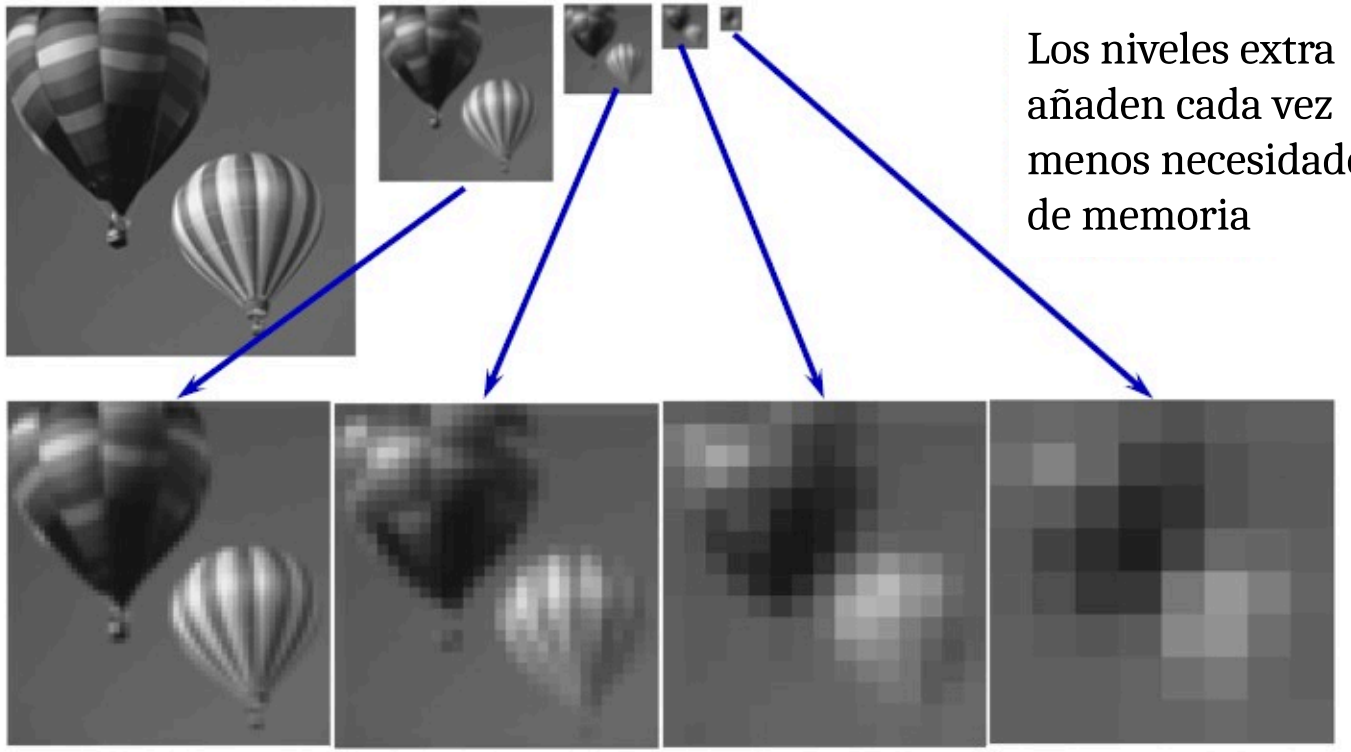
“Esqueletización”
(Medial axis)

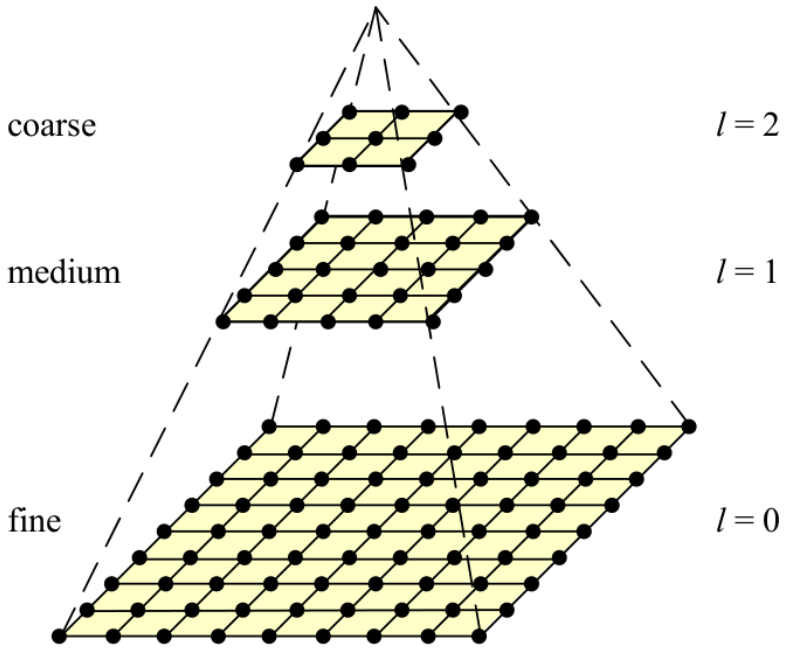


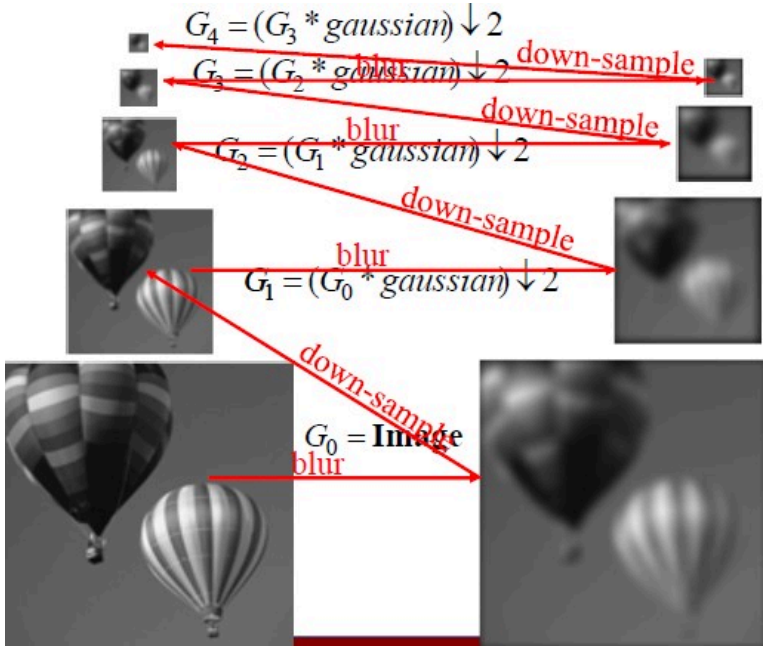
Distancia entre
contornos

Procesamiento de imagen para visión artificial
Pirámides de imagen

- Reducción iterativa del tamaño de la imagen.
  - P.e. para realizar búsquedas multiescala.
  - Habitual también en operaciones de *pooling* en CNNs.
- Suavizar antes de submuestrear → evita *aliasing* (https://es.wikipedia.org/wiki/Aliasing)



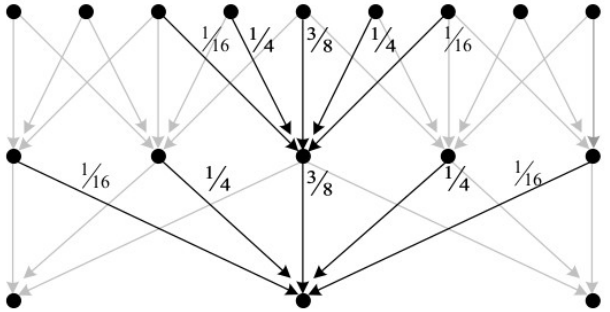




$$\frac{1}{16}\begin{array}{|c|c|c|c|c|}\hline 1 & 4 & 6 & 4 & 1 \\ \hline\end{array}$$

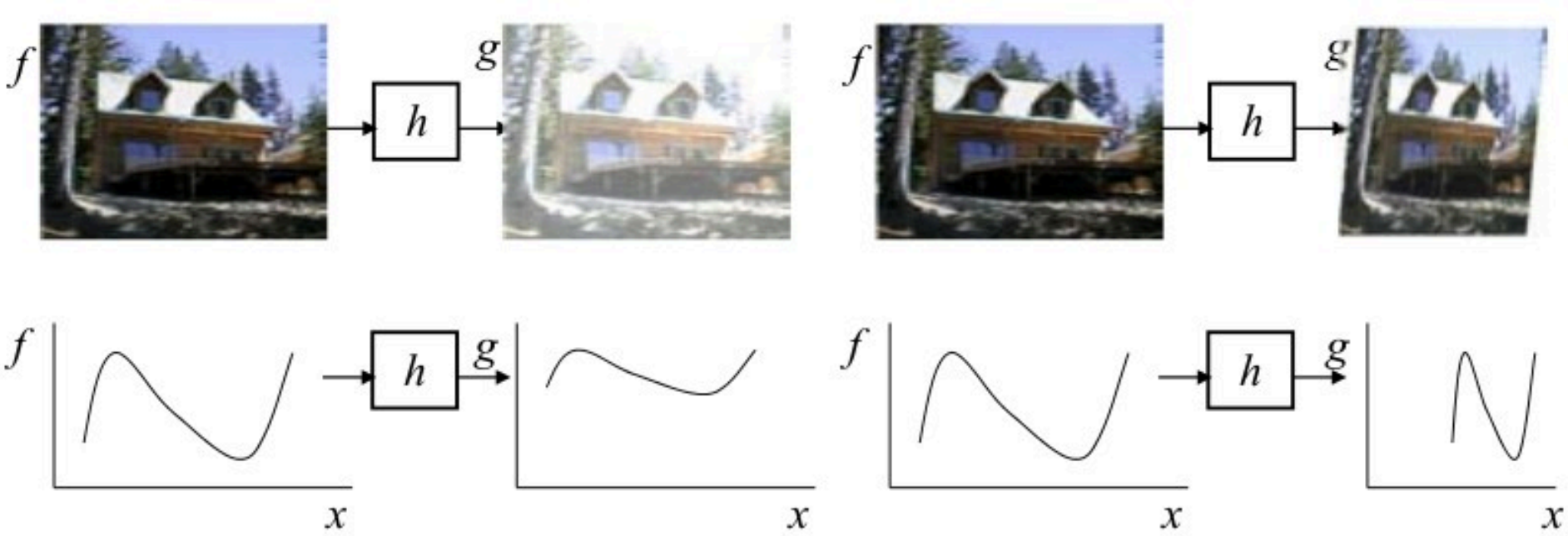
Típico filtro *five-tap*

Importante para SIFT (Tema 4)



- Se realiza una modificación del dominio de la imagen, en lugar del rango (*warping*):

$$g(\boldsymbol{x}) = f(\boldsymbol{h}(\boldsymbol{x})) \quad \text{en lugar de} \quad g(\boldsymbol{x}) = h(f(\boldsymbol{x}))$$



Transformación de RANGO

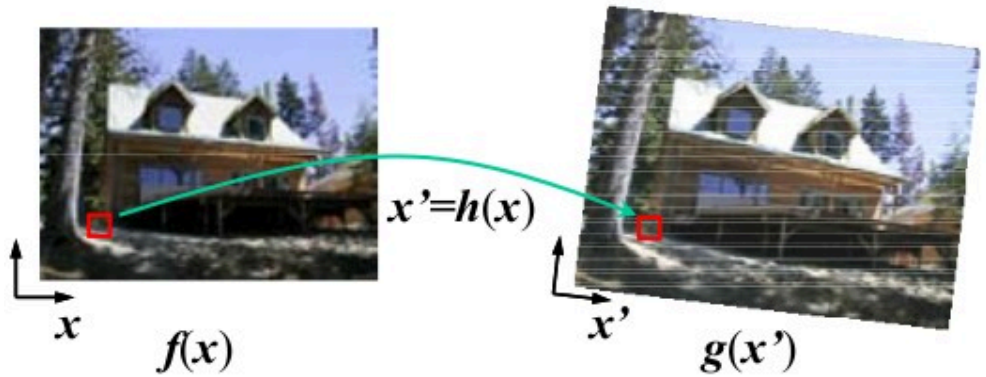
Transformación de DOMINIO

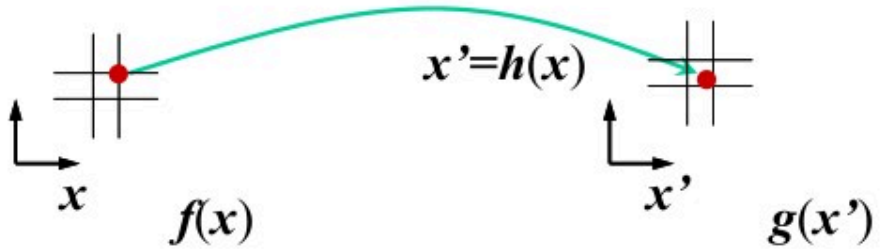
- Procedimiento de *warping* ingenuo (*forward warping*):

procedure *forwardWarp*$(f, \boldsymbol{h}, \textbf{out}\ g)$:

For every pixel $\boldsymbol{x}$ in $f(\boldsymbol{x})$

1. Compute the destination location $\boldsymbol{x}' = \boldsymbol{h}(\boldsymbol{x})$.
2. Copy the pixel $f(\boldsymbol{x})$ to $g(\boldsymbol{x}')$.



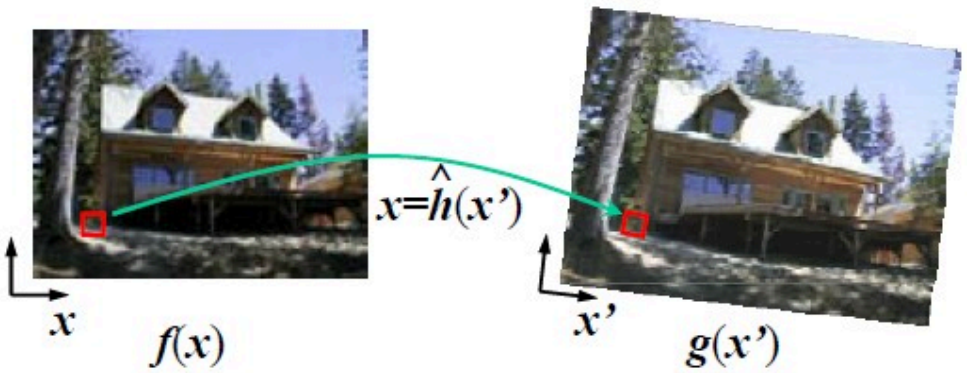


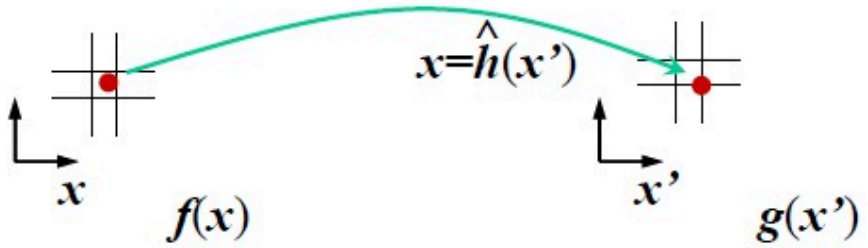
- Procedimiento de *warping* inverso, óptimo para evitar huecos:

procedure *inverseWarp*$(f, h,$ **out** $g)$:

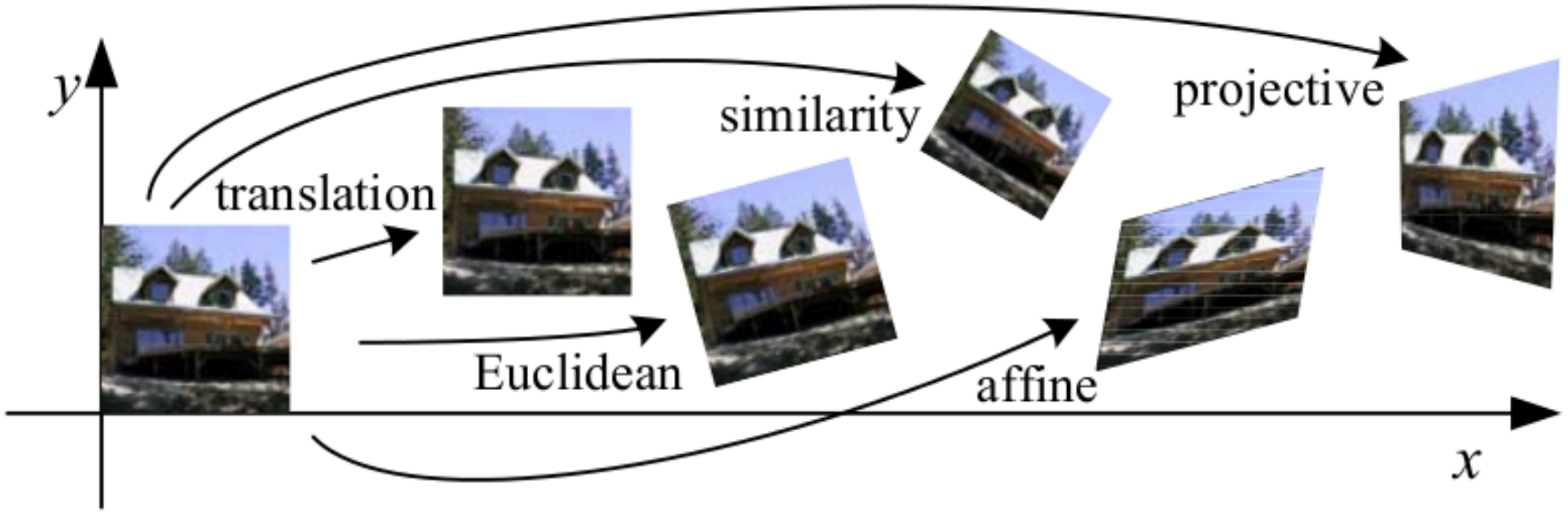
For every pixel x' in $g(x')$

1. Compute the source location $x = \hat{h}(x')$
2. Resample $f(x)$ at location x and copy to $g(x')$

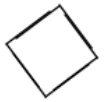
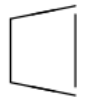




- Repertorio básico de transformaciones geométricas (*warpings*) 2D:

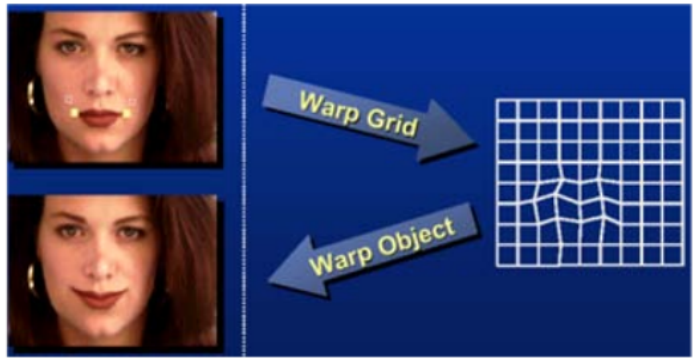


- Jerarquía de transformaciones 2D con coordenadas homogéneas $(x, y, 1)^T$.
- La coordenada adicional generada por la homografía H divide a las otras dos.
- A menudo se extienden matrices 2×3 con una tercera fila $(0, 0, 1)$ para no tratar casos especiales.

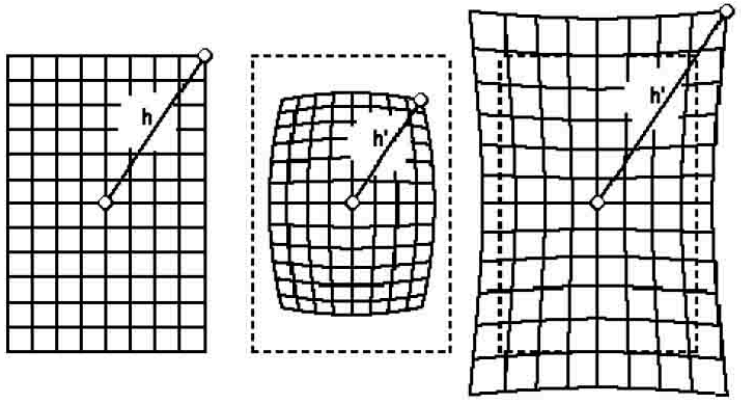
Transformation	Matrix	# DoF	Preserves	Icon
translation	$\left[\boldsymbol{I} \mid \boldsymbol{t} \right]_{2\times3}$	2	orientation	
rigid (Euclidean)	$\left[\boldsymbol{R} \mid \boldsymbol{t} \right]_{2\times3}$	3	lengths	
similarity	$\left[s\boldsymbol{R} \mid \boldsymbol{t} \right]_{2\times3}$	4	angles	
affine	$\left[\boldsymbol{A} \right]_{2\times3}$	6	parallelism	
projective	$\left[\tilde{\boldsymbol{H}} \right]_{3\times3}$	8	straight lines	

Procesamiento de imagen para visión artificial
Otras transformaciones geométricas

- Transformaciones basadas en mallas (*meshes*):



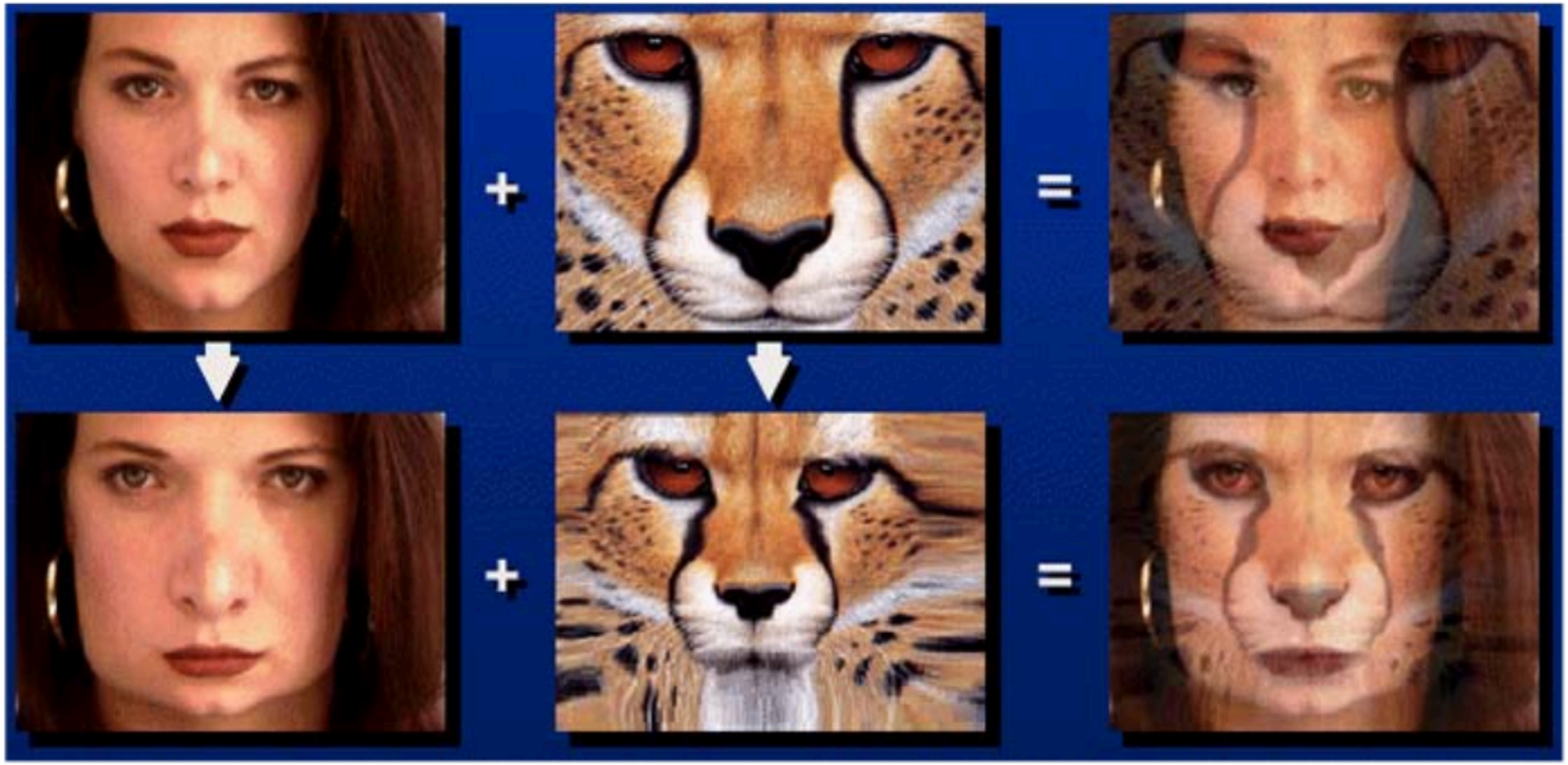
- Ejemplo: **corrección de la distorsión radial**:
  - Cada píxel se mapea a una posición en la línea que lo une con el centro de la imagen, desplazándose una distancia proporcional al radio según un polinomio de coeficientes ajustables):



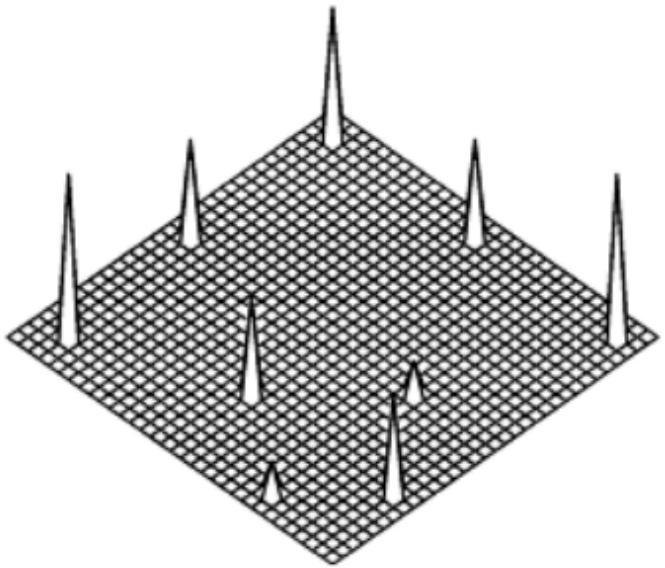


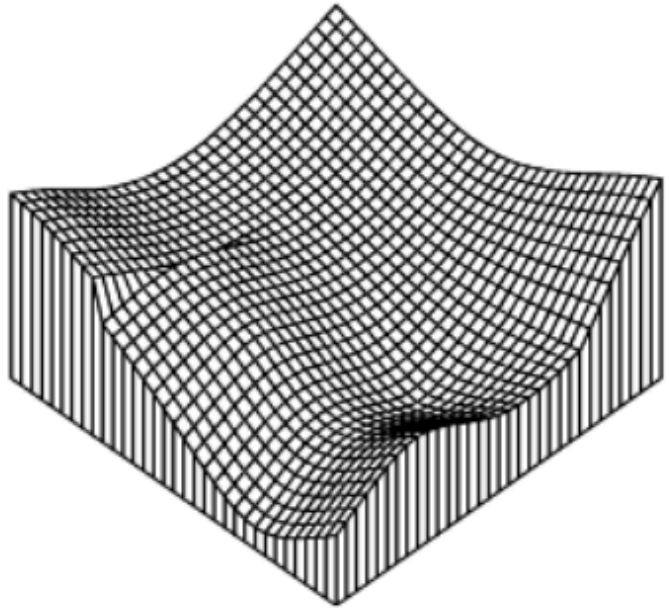


- *Warping* combinado con *blending* $\rightarrow$ *morphing* más suave:



- Inferir la imagen de salida optimizando algún criterio sobre la de entrada, dadas unas restricciones.
  - **Ejemplo**: interpolación "suave", respetando ciertos valores prefijados:





Material para profundizar

- Ejemplo de interpolación:
  - Reconstrucción de una imagen a partir de sus bordes (método de Elder-Zucker).
  - Proceso de difusión (ecuación del calor): a cada píxel de salida se asigna la media de sus vecinos, y se itera.
  - En realidad, minimiza el laplaciano de la imagen.



Original



Bordes (Canny)



Información relevante (bordes+grises)



Reconstrucción final

Procesamiento de imagen para visión artificial
Grupo 4: Optimización global sobre la imagen



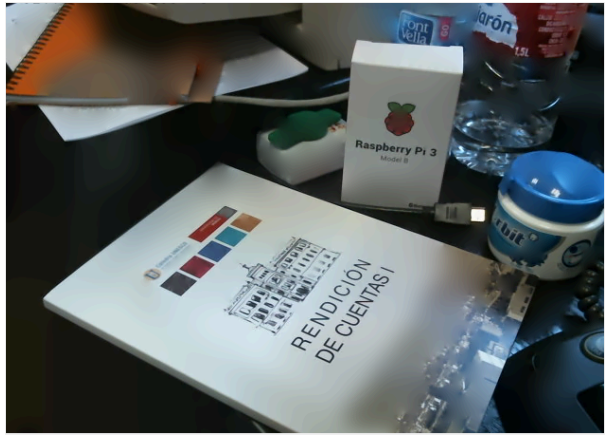
1. Canny



2. Dilatación + muestreo color



3. Rellenado color con DT



4. Reconstrucción final

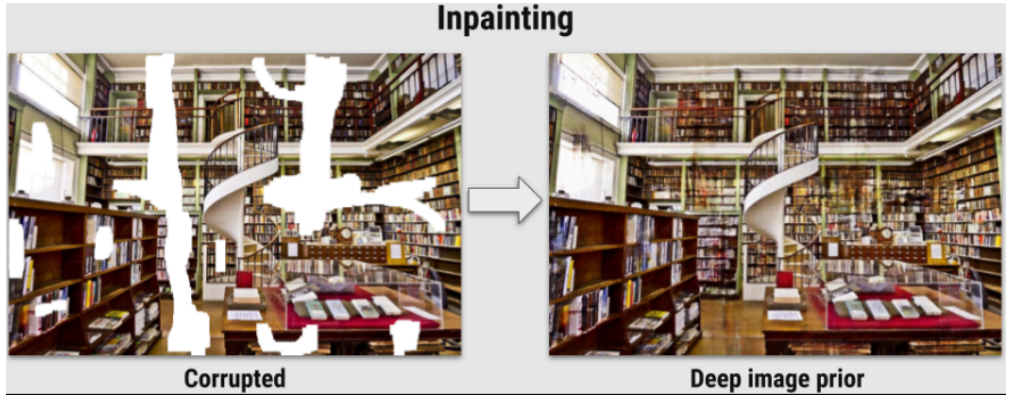
- *Denoising* e *inpainting*:
  - Clásico





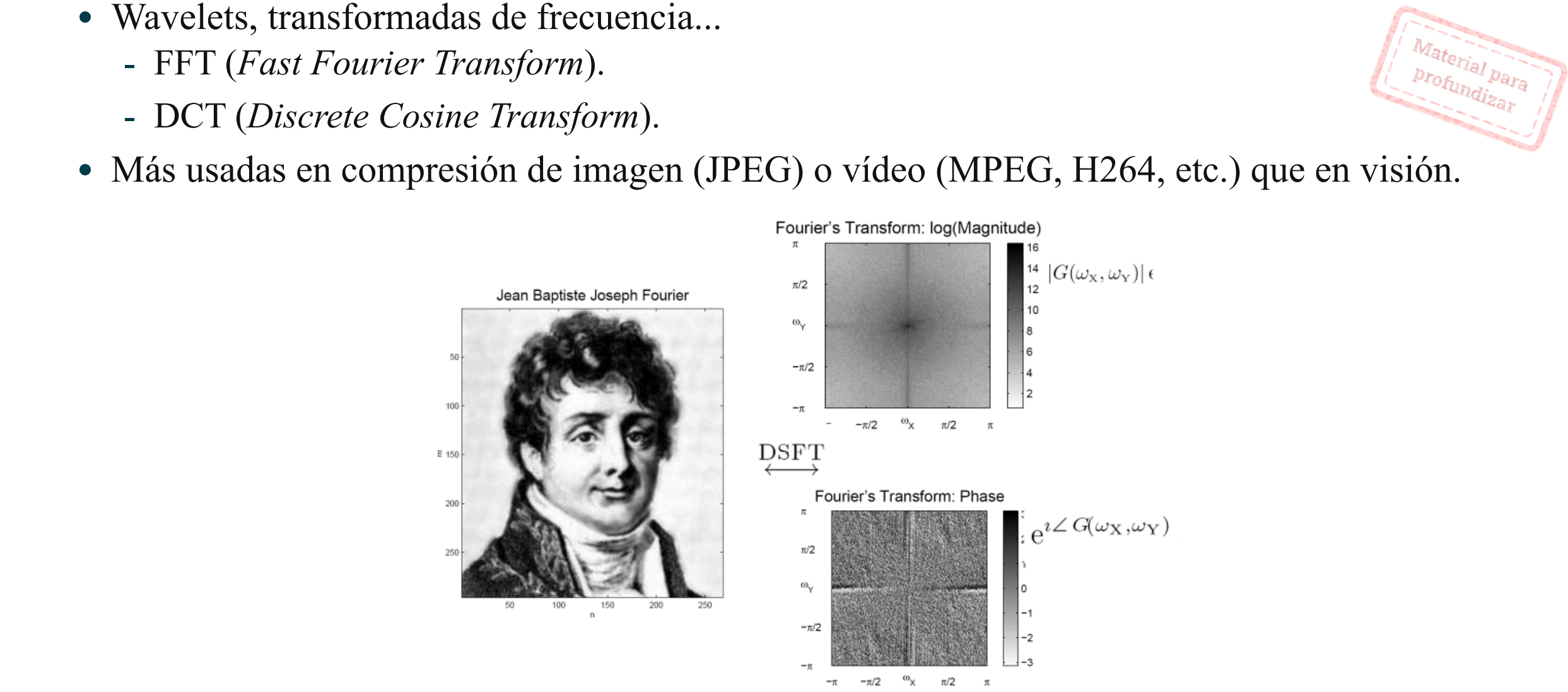


  - Con CNNs:



Procesamiento de imagen para visión artificial
Otras operaciones sobre imágenes

- Wavelets, transformadas de frecuencia...
  - FFT (*Fast Fourier Transform*).
  - DCT (*Discrete Cosine Transform*).
- Más usadas en compresión de imagen (JPEG) o vídeo (MPEG, H264, etc.) que en visión.



- NumPy, scikit-image y OpenCV proporcionan prácticamente todas las operaciones estudiadas.
- Muchas ejecutan código nativo optimizado, con vectorización SIMD y bibliotecas como Intel IPP / Intel TBB (ambas usadas internamente por OpenCV).
- También hay alternativas para GPU: CUDA, OpenCL, CuPy, etc.
- Un *wrapper* propio permite integrar rutinas C/C++ en Python:
  - Notebook 05: histograma y Otsu con `ctypes`.
  - Cuidar tipos, continuidad de los arrays, memoria y compilación reproducible.
  - Verificar resultados y medir: escribir C/C++ **no siempre** garantiza mayor velocidad.

Referencias

- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2022: ↲
- Principal: notebook `04_ImageProcessingForCV.ipynb` de prácticas.
- Integración de C: notebook `05_OtsuContoursHomog2DWrappersC.ipynb`.
- Notebooks adicionales:
  - Primitivas gráficas, RGB ↔ gris, superficie imagen: ↲
  - Espacios de color: ↲
  - Histogramas, ecualización: ↲
  - Cuantización de color: ↲
  - Transformaciones de dominio y distorsión radial con NumPy: ↲
- Tutorial Python+OpenCV: ↲
  - Image processing, color spaces, geometric transformations, thresholding, smoothing, morphology, gradients, pyramids, contours, histograms, Fourier/cosine transforms, template matching, GrabCut.